ECE181: Introduction to Python

Comprehensive University Question Bank & Solved Problems

LPU Engineering Academic Faculty

Edition 2026–2027

Contents

1 Unit 1: Introduction, Variables, Expressions and Statements **1**
- 1.1 Section A: Multiple Choice Practice Questions (60 Curated Problems) 1
- 1.2 Section B: Comprehensive Subjective Problem with Detailed Analytical Rubric 9

2 Unit 2: Conditionals, Iterations, Functions and Recursion **10**
- 2.1 Section A: Multiple Choice Practice Questions (60 Curated Problems) 10
- 2.2 Section B: Comprehensive Subjective Problem with Detailed Analytical Rubric 21

3 Unit 3: Strings, Lists, Tuples and Dictionaries **22**
- 3.1 Section A: Multiple Choice Practice Questions (60 Curated Problems) 22
- 3.2 Section B: Comprehensive Subjective Problem with Detailed Analytical Rubric 33

4 Unit 4: Array Operations using NumPy **34**
- 4.1 Section A: Multiple Choice Practice Questions (60 Curated Problems) 34
- 4.2 Section B: Comprehensive Subjective Problem with Detailed Analytical Rubric 43

5 Unit 5: Handling Data with Pandas and Data Visualisation with Matplotlib **44**
- 5.1 Section A: Multiple Choice Practice Questions (60 Curated Problems) 44
- 5.2 Section B: Comprehensive Subjective Problem with Detailed Analytical Rubric 53

6 Unit 6: Real-World Applications (OpenCV, Pygame, Web Scraping, Toolboxes) **54**
- 6.1 Section A: Multiple Choice Practice Questions (60 Curated Problems) 54
- 6.2 Section B: Comprehensive Subjective Problem with Detailed Analytical Rubric 63

Chapter 1

Unit 1: Introduction, Variables, Expressions and Statements

1.1 Section A: Multiple Choice Practice Questions (60 Curated Problems)

This section contains 60 rigorous multiple-choice questions extracted from university course examination assessments. Review each question, test edge-case syntax, and verify code execution mechanics.

Q1. **[need for a programming language | Easy]**

What is the primary purpose of a programming language?

(a) To store data permanently on disk
(b) To increase the speed of the CPU
(c) To design computer hardware circuits
(d) To communicate instructions to a computer

Q2. **[introduction to python as a programming language | Easy]**

Python is best described as which type of programming language?

(a) A low-level assembly language
(b) A markup language for web pages
(c) A high-level, interpreted language
(d) A hardware description language

Q3. **[introduction to python as a programming language | Easy]**

Who is credited with creating the Python programming language?

(a) Dennis Ritchie
(b) James Gosling
(c) Guido van Rossum
(d) Bjarne Stroustrup

Q4. **[programming errors and debugging | Easy]**

Which type of error occurs when Python code violates the rules of the language, such as a missing colon?

(a) Runtime error
(b) Semantic error
(c) Logical error
(d) Syntax error

Q5. **[programming errors and debugging | Easy]**

What is the process of finding and fixing errors in a program called?

(a) Debugging
(b) Compiling
(c) Executing

(d) Formatting

Q6. [identifiers | Easy]

Which of the following is a valid Python identifier?

(a) class
(b) 2ndValue
(c) _myVar
(d) my-var

Q7. [identifiers | Easy]

Which statement about Python identifiers is true?

(a) They are case-sensitive
(b) They may contain spaces
(c) They ignore uppercase letters
(d) They can start with a digit

Q8. [variables | Easy]

What is a variable in Python?

(a) A fixed value that never changes
(b) A built-in function name
(c) A type of loop structure
(d) A named reference to a stored value

Q9. [assignment statements | Easy]

What does the statement `x = 5` do in Python?

(a) Prints the value 5
(b) Checks if x equals 5
(c) Assigns the value 5 to the variable x
(d) Compares x with 5

Q10. [expressions | Easy]

Which of the following is an example of an expression in Python?

(a) 3 +
(b) while True:
(c) if x:
(d) def foo():

Q11. [named constant | Easy]

By convention, how are named constants usually written in Python?

(a) In all lowercase letters
(b) In all uppercase letters
(c) In camelCase
(d) With a leading digit

Q12. [simultaneous assignment | Easy]

What will be the values of `a` and `b` after `a, b = 1, 2`?

(a) both a and b are 2
(b) both a and b are 1
(c) a is 2 and b is 1
(d) a is 1 and b is 2

Q13. [simultaneous assignment | Easy]

Which single statement correctly swaps the values of `x` and `y` in Python?

(a) x == y, y == x
(b) swap(x, y)
(c) x, y = y, x
(d) x = y = x

Q14. [boolean types | Easy]

Which of the following are the two Boolean values in Python?

(a) true and false
(b) True and False
(c) 1 and 0 only
(d) Yes and No

Q15. [numeric data types | Easy]

Which numeric data type represents whole numbers in Python?

(a) complex
(b) str
(c) float
(d) int

Q16. [numeric data types | Easy]

What is the data type of the value `3.14` in Python?

(a) complex
(b) float
(c) bool
(d) int

Q17. [operators | Easy]

Which operator is used for exponentiation in Python?

(a) //
(b) ^
(c) %
(d) **

Q18. [operator precedence and associativity | Easy]

What is the result of the expression `2 + 3 * 4` in Python?

(a) 14
(b) 20
(c) 24
(d) 11

Q19. [augmented assignment operators | Easy]

What does the statement `x += 3` mean in Python?

(a) x = 3
(b) x = x + 3
(c) x = x * 3
(d) x = 3 - x

Q20. [type conversion and rounding | Easy]

What does the function `int(4.9)` return in Python?

(a) 4
(b) 5
(c) 4.9
(d) 0

Q21. [numeric data types | Medium]

What is the data type of the result of the expression `7 / 2` in Python 3?

(a) float
(b) int
(c) decimal
(d) complex

Q22. [operators | Medium]

What is the output of `17 % 5` in Python?

(a) 2
(b) 12
(c) 3
(d) 5

Q23. [operator precedence and associativity | Medium]

What is the value of the expression `2 + 3 * 4 ** 2`?

(a) 400
(b) 50
(c) 80
(d) 196

Q24. [operator precedence and associativity | Medium]

What does the expression `2 ** 3 ** 2` evaluate to in Python?

(a) 512
(b) 256
(c) 64
(d) 128

Q25. [augmented assignment operators | Medium]

After the code `x = 10` followed by `x //= 3`, what is the value of `x`?

(a) 3.33
(b) 3
(c) 4
(d) 1

Q26. [simultaneous assignment | Medium]

After executing `a, b = 5, 10` and then `a, b = b, a`, what are the values of `a` and `b`?

(a) a = 10, b = 5
(b) a = 5, b = 5
(c) a = 10, b = 10
(d) a = 5, b = 10

Q27. [type conversion and rounding | Medium]

What is the output of `int(9.99)` in Python?

(a) 10
(b) 9.99
(c) 9
(d) 9.0

Q28. [type conversion and rounding | Medium]

What does `round(2.5)` return in Python 3?

(a) 2
(b) 2.0
(c) 2.5
(d) 3

Q29. [boolean types | Medium]

What is the result of the expression `True + True + False` in Python?

(a) 2
(b) TypeError
(c) True
(d) 1

Q30. [boolean types | Medium]

What is the output of `bool(0.0) or bool("")`?

(a) True
(b) False
(c) 0.0
(d) None

Q31. [identifiers | Medium]

Which of the following is a valid Python identifier?

(a) _total_2
(b) 2total
(c) total 2
(d) total-2

Q32. [identifiers | Medium]

Why does the statement `class = 5` raise an error in Python?

(a) `class` is a reserved keyword
(b) integers cannot be assigned to names
(c) `class` must be capitalized
(d) the value must be a string

Q33. [programming errors and debugging | Medium]

A program runs without crashing but produces an incorrect average. What type of error is this?

(a) Runtime error
(b) Logical error
(c) Indentation error
(d) Syntax error

Q34. [programming errors and debugging | Medium]

Dividing by zero in `x = 5 / 0` produces which kind of error?

(a) Runtime error
(b) Syntax error
(c) Semantic error
(d) Logical error

Q35. [expressions | Medium]

What is the value of the expression `10 - 4 - 2` in Python?

(a) 0
(b) 6
(c) 8
(d) 4

Q36. [assignment statements | Medium]

What happens when the following runs: `x = 5` then `x = x + "3"`?

(a) x becomes 8
(b) TypeError is raised
(c) x becomes "53"
(d) x becomes 53

Q37. [named constant | Medium]

By convention, how should a named constant like the value of Pi be written in Python?

(a) pi
(b) Pi
(c) PI
(d) *pi*

Q38. [variables | Medium]

After `a = 3` and `b = a` and then `a = 7`, what is the value of `b`?

(a) 3

(b) 7
(c) 10
(d) None

Q39. [need for a programming language | Medium]
Why do we use high-level programming languages instead of machine code directly?
(a) They are easier for humans to read and write
(b) They run faster than machine code
(c) They require no translation to execute
(d) They use only binary digits

Q40. [introduction to python as a programming language | Medium]
Which statement best describes Python as a programming language?
(a) It is an interpreted, dynamically typed language
(b) It requires variable types to be declared
(c) It converts code directly to machine code
(d) It is a compiled, statically typed language

Q41. [operator precedence and associativity | Hard]
What is the result of evaluating `2 ** 3 ** 2` in Python?
(a) 81
(b) 36
(c) 512
(d) 64

Q42. [numeric data types | Hard]
What does `0.1 + 0.2 == 0.3` evaluate to in Python, and why?
(a) `False`, due to floating-point representation error
(b) `True`, since decimals are exact
(c) `True`, after automatic rounding
(d) `False`, because `==` is invalid for floats

Q43. [type conversion and rounding | Hard]
What is the output of `round(2.5) + round(3.5)`?
(a) 5
(b) 8
(c) 7
(d) 6

Q44. [simultaneous assignment | Hard]
After executing `a, b = 1, 2` then `a, b = b, a + b`, what are the values of `a` and `b`?
(a) `a = 3, b = 2`
(b) `a = 1, b = 3`
(c) `a = 2, b = 3`
(d) `a = 2, b = 4`

Q45. [boolean types | Hard]
What is the result of `True + True + False` in Python?
(a) 1
(b) `TypeError`
(c) 2
(d) `True`

Q46. [operators | Hard]
What is the value of `-7 // 2` in Python?
(a) -4
(b) -3

(c) -3.5
(d) 3

Q47. [operators | Hard]
What does `-7 % 3` evaluate to in Python?
(a) -2
(b) 2
(c) 1
(d) -1

Q48. [expressions | Hard]
What is the output of `2 + 3 * 4 ** 2 // 5 - 1`?
(a) 11
(b) 10
(c) 9
(d) 8

Q49. [augmented assignment operators | Hard]
Given `x = 5`, what is the value of `x` after `x //= 2; x **= 2; x -= 1`?
(a) 5
(b) 8
(c) 3
(d) 4

Q50. [identifiers | Hard]
Which of the following is a valid Python identifier?
(a) `class`
(b) `total-sum`
(c) `_total2`
(d) `2total`

Q51. [type conversion and rounding | Hard]
What is the output of `int("10", 2) + int("10")`?
(a) 22
(b) 4
(c) 20
(d) 12

Q52. [numeric data types | Hard]
What is the type and value of `5 / 5` in Python 3?
(a) `float`, `1`
(b) `int`, `1.0`
(c) `float`, `1.0`
(d) `int`, `1`

Q53. [programming errors and debugging | Hard]
A program runs without crashing but produces incorrect output. What type of error is this?
(a) Logical (semantic) error
(b) Syntax error
(c) Runtime exception
(d) Indentation error

Q54. [expressions | Hard]
What is the result of `not True == False` in Python?
(a) `False`
(b) `None`
(c) `TypeError`

(d) `True`

Q55. [operators | Hard]

What does `5 & 3 | 2` evaluate to?

(a) 1
(b) 7
(c) 3
(d) 2

Q56. [assignment statements | Hard]

After `x = y = [1, 2]; x.append(3)`, what is the value of `y`?

(a) `[1, 2, 3]`
(b) `[3]`
(c) `NameError`
(d) `[1, 2]`

Q57. [named constant | Hard]

What is true about a named constant like `PI = 3.14159` in Python?

(a) It must be declared with a `const` keyword
(b) It is stored in read-only memory automatically
(c) It cannot be reassigned once defined
(d) It is a convention only; Python does not enforce immutability

Q58. [type conversion and rounding | Hard]

What is the output of `round(3.14159, 2)` and its type?

(a) `3.14`, `str`
(b) `3.14`, `float`
(c) `3.15`, `float`
(d) `3`, `int`

Q59. [boolean types | Hard]

What does the expression `0 or "" or [] or "Python"` return?

(a) `True`
(b) `[]`
(c) `"Python"`
(d) `0`

Q60. [introduction to python as a programming language | Hard]

Which statement best describes Python's execution model?

(a) Source code is compiled to bytecode, then run by an interpreter (virtual machine)
(b) Source code is translated to assembly and linked into an executable
(c) Source code is compiled directly to native machine code before running
(d) Source code is executed line-by-line with no intermediate representation

1.2 Section B: Comprehensive Subjective Problem with Detailed Analytical Rubric

Problem 1 **[Subjective Analytical Problem — 10 Marks]**

1 Design and implement a robust Python algorithm to compute compound interest and loan amortization with exact rounding and error handling:

(a) Accept principal amount P, annual interest rate r (percentage), loan tenure t (years), and compounding frequency n per year.

(b) Compute future compound value $A = P\left(1 + \frac{r}{100n}\right)^{nt}$ and effective annual rate $\text{EAR} = \left(1 + \frac{r}{100n}\right)^{n} - 1$.

(c) Validate inputs against type errors and domain boundaries ($P > 0, r > 0, t > 0, n \geq 1$).

(d) Format and round outputs strictly to 2 decimal places using Python string formatting.

Step-by-Step Analytical & Algorithmic Solution:

Algorithmic Formulation and Implementation:

```
def calculate_loan_metrics(principal, rate, years, freq):
  if not (principal > 0 and rate > 0 and years > 0 and freq >= 1):
    raise ValueError("Parameters must strictly satisfy P>0, r>0, t>0, n>=1")

  r_dec = rate / 100.0
  amount = principal * ((1.0 + r_dec / freq) ** (freq * years))
  ear = ((1.0 + r_dec / freq) ** freq) - 1.0
  compound_interest = amount - principal

  return {
    "maturity_amount":  round(amount, 2),
    "total_interest":  round(compound_interest, 2),
    "effective_rate_pct":  round(ear * 100.0, 4)
  }

# Test execution:
res = calculate_loan_metrics(100000, 8.5, 5, 12)
print(f"Maturity:  $res['maturity_amount']:.2f, Effective Rate:  {res['effective_rate_pct']}%")
```

Complexity and Edge Analysis: The arithmetic operations execute in $\mathcal{O}(1)$ time with $\mathcal{O}(1)$ auxiliary memory. Floor division and simultaneous type checks prevent zero division or negative base floating-point artifacts.

- **Algorithmic Formulation (3 Marks):** Correct problem decomposition, function signatures, and parameter documentation.
- **Syntactic Rigor & Implementation (4 Marks):** Valid Python syntax, proper error handling, defensive input validation, and avoidance of anti-patterns.
- **Complexity & Edge Analysis (3 Marks):** Accurate time and space asymptotic analysis $\mathcal{O}(n)$ and testing edge conditions (null inputs, zero division, empty collections).

Chapter 2

Unit 2: Conditionals, Iterations, Functions and Recursion

2.1 Section A: Multiple Choice Practice Questions (60 Curated Problems)

This section contains 60 rigorous multiple-choice questions extracted from university course examination assessments. Review each question, test edge-case syntax, and verify code execution mechanics.

Q1. [if statement | Easy]

Which keyword is used to begin a conditional statement in Python?

(a) `when`
(b) `if`
(c) `check`
(d) `cond`

Q2. [two way if - else | Easy]

In a two-way `if-else` statement, when does the `else` block execute?

(a) When the `if` condition is false
(b) When the `if` condition is true
(c) Only when the condition evaluates to a number greater than zero, considering all edge cases carefully
(d) Always, regardless of the condition

Q3. [nested if and multi-way if-elif-else statement | Easy]

Which keyword is used for a multi-way branch to test additional conditions in Python?

(a) `elseif`
(b) `elsif`
(c) `elif`
(d) `orif`

Q4. [for loop | Easy]

What does the `range(5)` function generate when used in a `for` loop?

(a) Numbers `0, 1, 2, 3, 4`
(b) Numbers `0, 1, 2, 3, 4, 5`
(c) Numbers `1, 2, 3, 4, 5`
(d) Numbers `1, 2, 3, 4`

Q5. [while loop | Easy]

A `while` loop in Python continues to execute as long as its condition is:

(a) `True`
(b) `False`
(c) `zero`

(d) `None`

Q6. [break and continue | Easy]

What does the `break` statement do inside a loop?

(a) Pauses the loop temporarily until resumed by the interpreter automatically
(b) Restarts the loop from the beginning
(c) Skips to the next iteration of the loop
(d) Exits the loop immediately

Q7. [break and continue | Easy]

What does the `continue` statement do inside a loop?

(a) Stops the program execution
(b) Exits the loop completely
(c) Repeats the current iteration
(d) Skips the rest of the current iteration and moves to the next

Q8. [nested loops | Easy]

What is a nested loop?

(a) A loop that runs backwards
(b) A loop that never ends
(c) A loop with only one iteration
(d) A loop placed inside another loop

Q9. [random numbers | Easy]

Which module in Python is commonly used to generate random numbers?

(a) `numbers`
(b) `random`
(c) `rand`
(d) `math`

Q10. [random numbers | Easy]

Which function returns a random integer between two given values (inclusive)?

(a) `random.randint(a, b)`
(b) `random.range()`
(c) `random.choice()`
(d) `random.random()`

Q11. [function calls | Easy]

How do you call a function named `greet` in Python?

(a) `greet()`
(b) `function greet`
(c) `greet[]`
(d) `call greet`

Q12. [type conversion and coercion | Easy]

Which function converts a string `"25"` into an integer?

(a) `int("25")`
(b) `chr(25)`
(c) `float("25")`
(d) `str(25)`

Q13. [type conversion and coercion | Easy]

What is the result of `float(7)` in Python?

(a) `70`
(b) `7.0`
(c) `"7"`
(d) `7`

Q14. [math functions | Easy]

Which function from the `math` module returns the square root of a number?

(a) `math.sqrt()`
(b) `math.pow()`
(c) `math.root()`
(d) `math.square()`

Q15. [math functions | Easy]

What does `math.ceil(4.2)` return?

(a) `4`
(b) `4.0`
(c) `4.2`
(d) `5`

Q16. [adding new function | Easy]

Which keyword is used to define a new function in Python?

(a) `def`
(b) `func`
(c) `define`
(d) `function`

Q17. [parameters and argument | Easy]

In the function definition `def add(x, y):`, what are `x` and `y` called?

(a) Parameters
(b) Keywords
(c) Variables
(d) Arguments

Q18. [recursion and its use | Easy]

What is recursion in programming?

(a) A function that calls itself
(b) A loop that runs forever
(c) A function with no parameters
(d) A function that returns nothing

Q19. [recursion and its use | Easy]

What is required in every recursive function to prevent infinite recursion?

(a) A print statement that displays the current value at each level of the recursive call chain
(b) A base case
(c) A global variable
(d) An infinite loop

Q20. [conditional expressions | Easy]

What is the value of the conditional expression `"Yes" if 5 > 3 else "No"`?

(a) `"No"`
(b) `"Yes"`
(c) `True`
(d) `False`

Q21. [conditional expressions | Medium]

What is the value assigned to `result` after executing the following code?

```
 x = 7
result = "even" if x % 2 == 0 else "odd"
```

(a) `7`
(b) `True`
(c) `"even"`

(d) `"odd"`

Q22. [if statement | Medium]

How many times will `"Hi"` be printed?

```
n = 10
if n > 5:
if n < 20:
print("Hi")
```

(a) `0`
(b) `1`
(c) `10`
(d) `2`

Q23. [two way if - else | Medium]

What does the following code output?

```
age = 18
if age >= 18:
print("Adult")
else:
print("Minor")
```

(a) `Adult`
(b) Nothing is printed
(c) `AdultMinor`
(d) `Minor`

Q24. [nested if and multi-way if-elif-else statement | Medium]

What is printed by this code?

```
marks = 75
if marks >= 90:
print("A")
elif marks >= 70:
print("B")
elif marks >= 50:
print("C")
else:
print("F")
```

(a) `F`
(b) `B`
(c) `C`
(d) `A`

Q25. [for loop | Medium]

What is the output of the following loop?

```
total = 0
for i in range(1, 5):
total += i
print(total)
```

(a) `5`
(b) `6`
(c) `10`
(d) `15`

Q26. [while loop | Medium]

How many times does the `while` loop body execute?

```
x = 16
count = 0
while x > 1:
```

```
x = x // 2
count += 1
print(count)
```

(a) 16
(b) 4
(c) 5
(d) 3

Q27. [nested loops | Medium]

What is the output of the following nested loops?

```
 count = 0
for i in range(3):
for j in range(2):
count += 1
print(count)
```

(a) 3
(b) 5
(c) 6
(d) 2

Q28. [break and continue | Medium]

What does this code print?

```
 for i in range(1, 6):
if i == 3:
continue
if i == 5:
break
print(i, end=" ")
```

(a) 1 2 3 4 5
(b) 1 2 4 5
(c) 1 2 4
(d) 1 2 3 4

Q29. [random numbers | Medium]

Which function call returns a random integer that could be any value in the set $1, 2, 3, 4, 5, 6$?

(a) `random.range(1, 6)`
(b) `random.random(1, 6)`
(c) `random.randint(1, 6)`
(d) `random.randrange(1, 6)`

Q30. [function calls | Medium]

What is printed by the following code?

```
 def square(n):
return n * n

print(square(square(2)))
```

(a) 8
(b) 2
(c) 16
(d) 4

Q31. [type conversion and coercion | Medium]

What is the result of the following expression?

```
int("12") + int(3.9)
```

(a) 16
(b) "123"

(c) `15`
(d) `15.9`

Q32. [type conversion and coercion | Medium]

What is the output of the following code?

```
print(3 + True + 2.0)
```

(a) `5`
(b) `6`
(c) `6.0`
(d) `TypeError`

Q33. [math functions | Medium]

What does the following code print?

```
import math
print(math.ceil(4.1) + math.floor(4.9))
```

(a) `4`
(b) `9`
(c) `10`
(d) `8`

Q34. [math functions | Medium]

What is the value of the following expression?

```
import math
math.sqrt(math.pow(3, 2) + math.pow(4, 2))
```

(a) `25.0`
(b) `5.0`
(c) `12.0`
(d) `7.0`

Q35. [adding new function | Medium]

What does the following program print?

```
def greet(name):
print("Hello", name)

x = greet("Sam")
print(x)
```

(a) `None` only
(b) `Hello Sam` only
(c) `Hello Sam` then `Sam`
(d) `Hello Sam` then `None`

Q36. [parameters and argument | Medium]

What is the output of this code using default and keyword arguments?

```
def power(base, exp=2):
return base ** exp

print(power(3), power(2, exp=3))
```

(a) `8 9`
(b) `9 6`
(c) `6 8`
(d) `9 8`

Q37. [parameters and argument | Medium]

What does the following code print?

```
def info(a, b, c):
```

```
return a - b - c

print(info(c=1, a=10, b=3))
```

(a) 8
(b) 4
(c) 6
(d) 12

Q38. [recursion and its use | Medium]

What value does `fact(4)` return?

```
def fact(n):
if n <= 1:
return 1
return n * fact(n - 1)
```

(a) 4
(b) 24
(c) 12
(d) 10

Q39. [recursion and its use | Medium]

What is printed by the following recursive function call `mystery(3)`?

```
def mystery(n):
if n == 0:
return
print(n, end=" ")
mystery(n - 1)
```

(a) 0 1 2 3
(b) 3 2 1 0
(c) 3 2 1
(d) 1 2 3

Q40. [for loop | Medium]

What is the output of the following code that uses `range` with a step?

```
result = []
for i in range(10, 0, -3):
result.append(i)
print(result)
```

(a) [9, 6, 3]
(b) [10, 7, 4, 1]
(c) [10, 7, 4, 1, -2]
(d) [10, 7, 4]

Q41. [conditional expressions | Hard]

What is the output of the following code?

```
x = 0
y = 5
result = "A" if x else "B" if y else "C"
print(result)
```

(a) B
(b) A
(c) C
(d) Raises SyntaxError

Q42. [if statement | Hard]

Consider:

```
values = [0, '', None, [], 'False', 0.0]
```

```
count = 0
for v in values:
if v:
count += 1
print(count)
```

What is printed?

(a) 1
(b) 0
(c) 2
(d) 6

Q43. [two way if - else | Hard]

What does this function return for `f(-4)`?

```
 def f(n):
if n % 2 == 0:
return n // 2
else:
return 3 * n + 1
```

(a) 2
(b) -2
(c) -1
(d) -11

Q44. [nested if and multi-way if-elif-else statement | Hard]

What is the output?

```
 score = 85
if score >= 90:
grade = 'A'
elif score >= 80:
if score >= 85:
grade = 'B+'
else:
grade = 'B'
elif score >= 70:
grade = 'C'
else:
grade = 'F'
print(grade)
```

(a) C
(b) B
(c) A
(d) B+

Q45. [for loop | Hard]

What is the final value of `total`?

```
 total = 0
for i in range(2, 20, 3):
total += i
print(total)
```

(a) 45
(b) 51
(c) 63
(d) 57

Q46. [while loop | Hard]

How many times does the loop body execute?

```
 n = 100
```

```
count = 0
while n > 1:
n = n // 2
count += 1
print(count)
```

(a) 7
(b) 6
(c) 5
(d) 8

Q47. [nested loops | Hard]

What is printed?

```
 count = 0
for i in range(4):
for j in range(i):
count += 1
print(count)
```

(a) 16
(b) 12
(c) 10
(d) 6

Q48. [break and continue | Hard]

What is the output?

```
 result = []
for i in range(1, 10):
if i % 2 == 0:
continue
if i > 6:
break
result.append(i)
print(result)
```

(a) [1, 2, 3, 4, 5]
(b) [1, 3, 5]
(c) [1, 3, 5, 7]
(d) [1, 3, 5, 7, 9]

Q49. [break and continue | Hard]

What does this print?

```
 for i in range(3):
for j in range(3):
if j == 1:
break
print(i, j)
```

(a) Prints `0 0`, `1 0`, `2 0`
(b) Prints `0 0` only
(c) Prints `0 0`, `0 1`, `0 2`
(d) Prints all nine pairs

Q50. [random numbers | Hard]

Given `import random; random.seed(0)`, which statement is true about `random.randint(1, 6)`?

(a) It excludes the upper bound 6
(b) It returns values from 1 to 5 only
(c) It returns floats between 1 and 6
(d) It can return any integer from 1 to 6 inclusive

Q51. [random numbers | Hard]

What is the possible range of values produced by `random.randrange(2, 20, 4)`?

(a) 2, 6, 10, 14, 18
(b) 4, 8, 12, 16
(c) 2, 6, 10, 14, 18, 20
(d) Any integer from 2 to 20

Q52. [function calls | Hard]

What is the output?

```
 def g(x):
return x * 2

def h(x):
return x + 3

print(g(h(g(2))))
```

(a) 11
(b) 14
(c) 20
(d) 10

Q53. [type conversion and coercion | Hard]

What is the result of the following?

```
 print(int("10", 2) + int("10", 8) + int("10", 16))
```

(a) 30
(b) 10
(c) 26
(d) 42

Q54. [type conversion and coercion | Hard]

What is the output?

```
 print(int(True) + float("3.5") + int("7"))
```

(a) 10.5
(b) 10
(c) 11.5
(d) 11

Q55. [math functions | Hard]

What does the following evaluate to?

```
 import math
print(math.floor(math.sqrt(50)) + math.ceil(math.log2(10)))
```

(a) 10
(b) 11
(c) 12
(d) 13

Q56. [math functions | Hard]

What is the output?

```
 import math
print(math.gcd(48, 36), math.factorial(4) % 7)
```

(a) 12 3
(b) 6 4
(c) 12 4
(d) 6 3

Q57. [adding new function | Hard]

What is printed?

```
def make_counter():
count = 0
def inc():
nonlocal count
count += 1
return count
return inc

c = make_counter()
print(c(), c(), c())
```

(a) 1 1 1
(b) 1 2 3
(c) 0 1 2
(d) 3 3 3

Q58. [parameters and argument | Hard]

What is the output?

```
def f(a, b=[]):
b.append(a)
return b

print(f(1))
print(f(2))
print(f(3, []))
```

(a) [1] then [1, 2] then [1, 2, 3]
(b) [1] then [2] then [3]
(c) [1, 2, 3] repeated three times
(d) [1] then [1, 2] then [3]

Q59. [recursion and its use | Hard]

What does `mystery(4)` return?

```
def mystery(n):
if n <= 1:
return 1
return n * mystery(n - 1)
```

(a) 10
(b) 12
(c) 24
(d) 4

Q60. [recursion and its use | Hard]

How many times is `fib` called (including the initial call) when computing `fib(4)`?

```
def fib(n):
if n < 2:
return n
return fib(n - 1) + fib(n - 2)
```

(a) 5
(b) 9
(c) 7
(d) 11

2.2 Section B: Comprehensive Subjective Problem with Detailed Analytical Rubric

Problem 2 **[Subjective Analytical Problem — 10 Marks]**

1 Design a complete recursive and iterative implementation of the binary search algorithm and memoized Fibonacci generator with call-stack analysis:

(a) Implement `binary_search(arr, low, high, target)` recursively.

(b) Provide the formal recurrence relation for execution time and solve via Master Theorem.

(c) Implement `fibonacci_memo(n, cache=None)` to evaluate $F(n)$ in $\mathcal{O}(n)$ time and compare stack-depth against naive recursion $\mathcal{O}(2^n)$.

Step-by-Step Analytical & Algorithmic Solution:

Recursive Implementation & Stack Analysis:

```
def binary_search(arr, low, high, target):
  if low > high:
    return -1
  mid = low + (high - low) // 2
  if arr[mid] == target:
    return mid
  elif arr[mid] > target:
    return binary_search(arr, low, mid - 1, target)
  else:
    return binary_search(arr, mid + 1, high, target)

def fibonacci_memo(n, cache=None):
  if cache is None:
    cache = {0:  0, 1:  1}
  if n not in cache:
    cache[n] = fibonacci_memo(n - 1, cache) + fibonacci_memo(n - 2, cache)
  return cache[n]
```

Recurrence Relation: For binary search, $T(n) = T(n/2)+\mathcal{O}(1)$. By Master Theorem ($a = 1, b = 2, d = 0$), since $\log_2(1) = 0 = d$, $T(n) = \Theta(\log n)$. For memoized Fibonacci, each state $0 \dots n$ is visited exactly once, reducing exponential calls 2^n to linear time $\mathcal{O}(n)$ with auxiliary call stack depth n.

- **Algorithmic Formulation (3 Marks):** Correct problem decomposition, function signatures, and parameter documentation.
- **Syntactic Rigor & Implementation (4 Marks):** Valid Python syntax, proper error handling, defensive input validation, and avoidance of anti-patterns.
- **Complexity & Edge Analysis (3 Marks):** Accurate time and space asymptotic analysis $\mathcal{O}(n)$ and testing edge conditions (null inputs, zero division, empty collections).

Chapter 3

Unit 3: Strings, Lists, Tuples and Dictionaries

3.1 Section A: Multiple Choice Practice Questions (60 Curated Problems)

This section contains 60 rigorous multiple-choice questions extracted from university course examination assessments. Review each question, test edge-case syntax, and verify code execution mechanics.

Q1. [string a compound data type | Easy]

Why is a string considered a compound data type in Python?

(a) Because it is made up of smaller pieces called characters
(b) Because it always contains exactly one element that must be a letter of the alphabet from A to Z
(c) Because it can only store numbers
(d) Because it can be modified in place

Q2. [length | Easy]

Which function returns the number of characters in a string?

(a) `length()`
(b) `len()`
(c) `count()`
(d) `size()`

Q3. [accessing elements | Easy]

What is the index of the first character in a Python string?

(a) `0`
(b) `length`
(c) `1`
(d) `-1`

Q4. [string slices | Easy]

What is the result of `"python"[0:3]`?

(a) `"pyth"`
(b) `"pyt"`
(c) `"tho"`
(d) `"python"`

Q5. [string traversal | Easy]

Which loop is commonly used to traverse each character of a string?

(a) `if` statement
(b) `try` block
(c) `for` loop

(d) `with` statement

Q6. [comparison | Easy]

What does the expression `"apple" == "apple"` evaluate to?

(a) `False`
(b) `True`
(c) `None`
(d) `0`

Q7. [find function | Easy]

What does `"hello".find("l")` return?

(a) `3`
(b) `2`
(c) `1`
(d) `-1`

Q8. [find function | Easy]

What does the `find()` method return when the substring is not present?

(a) `None`
(b) `-1`
(c) An error is raised because the searched substring could not be located anywhere in the string
(d) `0`

Q9. [looping and counting | Easy]

What is a common use of a counter variable while looping through a string?

(a) To convert the string to uppercase
(b) To reverse the string
(c) To count occurrences of a character
(d) To delete characters from the string

Q10. [list values | Easy]

Which of the following correctly creates a list of three numbers?

(a) `(1, 2, 3)`
(b) `[1, 2, 3]`
(c) `{1, 2, 3}`
(d) `<1, 2, 3>`

Q11. [length | Easy]

What does `len([10, 20, 30, 40])` return?

(a) `3`
(b) `4`
(c) `5`
(d) `0`

Q12. [membership | Easy]

What does the expression `3 in [1, 2, 3]` evaluate to?

(a) `False`
(b) `3`
(c) `True`
(d) `None`

Q13. [operations | Easy]

What is the result of `[1, 2] + [3, 4]`?

(a) `[1, 2, [3, 4]]`
(b) `[4, 6]`
(c) `[1, 2, 3, 4]`
(d) `[3, 4, 1, 2]`

Q14. [slices | Easy]

What is the result of `[10, 20, 30, 40][1:3]`?

(a) `[20, 30, 40]`
(b) `[20, 30]`
(c) `[30, 40]`
(d) `[10, 20]`

Q15. [deletion | Easy]

Which keyword is used to remove an item from a list by its index?

(a) `del`
(b) `remove`
(c) `erase`
(d) `drop`

Q16. [list and for loops | Easy]

What is printed by `for x in [1, 2, 3]: print(x)`?

(a) The list `[1, 2, 3]`
(b) The sum `6`
(c) The length `3`
(d) Each element `1`, `2`, `3` on separate lines

Q17. [list parameters and nested list | Easy]

What is a nested list?

(a) A list with only numbers
(b) A list that contains one or more lists as elements
(c) A list stored in a file
(d) A list that cannot be changed

Q18. [tuples | Easy]

Which symbol pair is typically used to create a tuple?

(a) Parentheses `()`
(b) Square brackets `[]`
(c) Curly braces `{}`
(d) Angle brackets `<>`

Q19. [operations on tuples | Easy]

What key property makes tuples different from lists?

(a) Tuples are immutable and cannot be changed after creation
(b) Tuples use square brackets
(c) Tuples can only hold strings
(d) Tuples cannot be indexed

Q20. [creating dictionary | Easy]

Which of the following correctly creates a dictionary?

(a) `{"a", 1, "b", 2}`
(b) `["a": 1, "b": 2]`
(c) `{"a": 1, "b": 2}`
(d) `("a", 1, "b", 2)`

Q21. [string slices | Medium]

What is the output of the following code?

```
 s = "Programming"
print(s[2:8:2])
```

(a) `oai`
(b) `rgamn`
(c) `ogam`

(d) `orm`

Q22. [string traversal | Medium]

What does the following loop print?

```
word = "abc"
for ch in word:
print(ch * 2, end="")
```

(a) `aaabbbccc`
(b) `abcabc`
(c) `aabbcc`
(d) `abc`

Q23. [find function | Medium]

What is the result of the following code?

```
s = "banana"
print(s.find("na"))
```

(a) `4`
(b) `-1`
(c) `3`
(d) `2`

Q24. [comparison | Medium]

What is the output of the following comparison?

```
print("apple" < "banana")
```

(a) `0`
(b) `False`
(c) `True`
(d) `Error`

Q25. [looping and counting | Medium]

How many times does the letter `s` appear based on this code?

```
text = "mississippi"
count = 0
for c in text:
if c == "s":
count += 1
print(count)
```

(a) `3`
(b) `2`
(c) `4`
(d) `5`

Q26. [length | Medium]

What is the output of the following code?

```
s = "Hello, World!"
print(len(s))
```

(a) `11`
(b) `14`
(c) `13`
(d) `12`

Q27. [string a compound data type | Medium]

Which statement about Python strings is correct?

(a) Strings are stored as a list of integers
(b) Strings can only store alphabetic characters

(c) Strings are immutable, so `s[0] = "X"` raises an error
(d) Strings are mutable and support item assignment

Q28. [operations | Medium]

What is the output of the following code?

```
result = [1, 2] + [3, 4] * 2
print(result)
```

(a) `[1, 2, 6, 8]`
(b) `[1, 2, 3, 4, 1, 2, 3, 4]`
(c) `[4, 8, 3, 4]`
(d) `[1, 2, 3, 4, 3, 4]`

Q29. [slices | Medium]

What does the following code print?

```
nums = [10, 20, 30, 40, 50]
print(nums[-3:-1])
```

(a) `[20, 30]`
(b) `[40, 50]`
(c) `[30, 40, 50]`
(d) `[30, 40]`

Q30. [membership | Medium]

What is the output of the following code?

```
data = [[1, 2], [3, 4]]
print(3 in data)
```

(a) `[3, 4]`
(b) `True`
(c) `Error`
(d) `False`

Q31. [deletion | Medium]

What is the value of `nums` after this code runs?

```
nums = [1, 2, 3, 4, 5]
del nums[1:3]
print(nums)
```

(a) `[1, 3, 4, 5]`
(b) `[1, 4, 5]`
(c) `[2, 3, 4, 5]`
(d) `[1, 2, 5]`

Q32. [accessing elements | Medium]

What does the following code print?

```
matrix = [[1, 2, 3], [4, 5, 6]]
print(matrix[1][2])
```

(a) `5`
(b) `3`
(c) `6`
(d) `4`

Q33. [list and for loops | Medium]

What is the output of the following code?

```
nums = [1, 2, 3, 4]
total = 0
for n in nums:
total += n * n
```

```
print(total)
```

(a) 30
(b) 100
(c) 10
(d) 20

Q34. [list parameters and nested list | Medium]

What does the following code print?

```
 def modify(lst):
lst.append(99)

nums = [1, 2, 3]
modify(nums)
print(nums)
```

(a) [99, 1, 2, 3]
(b) [1, 2, 3, 99]
(c) [1, 2, 3]
(d) [99]

Q35. [tuples | Medium]

What happens when the following code is executed?

```
 t = (1, 2, 3)
t[0] = 10
print(t)
```

(a) Prints (10, 2, 3)
(b) Prints [10, 2, 3]
(c) Prints (1, 2, 3)
(d) Raises a TypeError

Q36. [operations on tuples | Medium]

What is the output of the following code?

```
 t = (1, 2) * 3
print(len(t))
```

(a) 5
(b) 6
(c) 2
(d) 3

Q37. [creating dictionary | Medium]

What is the output of the following code?

```
 d = dict([("a", 1), ("b", 2)])
print(d["b"])
```

(a) 1
(b) 2
(c) b
(d) ("b", 2)

Q38. [adding-modifying-retrieving dictionary values | Medium]

What does the following code print?

```
 d = {"x":  1}
d["y"] = 2
d["x"] = 5
print(d)
```

(a) {'x': 1, 'y': 2}
(b) {'x': 5}

(c) `{'x': 5, 'y': 2}`
(d) `{'x': 1, 'y': 2, 'x': 5}`

Q39. [deleting items | Medium]

What is the output of the following code?

```
d = {"a": 1, "b": 2, "c": 3}
del d["b"]
print(list(d.keys()))
```

(a) `['a', 'c']`
(b) `['b']`
(c) `['a', 'b', 'c']`
(d) `['a', 'c', 'b']`

Q40. [dictionary methods | Medium]

What is the output of the following code?

```
d = {"a": 1, "b": 2}
print(d.get("c", 0))
```

(a) `Error`
(b) `0`
(c) `None`
(d) `c`

Q41. [string slices | Hard]

What is the output of the following code?

```
s = "programming"
print(s[::-2])
```

(a) `gimrgm`
(b) `gimargo`
(c) `pormig`
(d) `gnimargorp`

Q42. [comparison | Hard]

Which expression evaluates to `True` in Python 3?

(a) `"Z" < "A"`
(b) `"apple" < "Apple"`
(c) `"ABC" > "abc"`
(d) `"Apple" < "apple"`

Q43. [find function | Hard]

What does the following print?

```
s = "mississippi"
print(s.find("ss", 3))
```

(a) `5`
(b) `2`
(c) `4`
(d) `-1`

Q44. [looping and counting | Hard]

What is the value of `count` after this loop?

```
s = "aAbBcCaA"
count = 0
for ch in s:
if ch.lower() == 'a':
count += 1
print(count)
```

(a) 4
(b) 2
(c) 3
(d) 8

Q45. [string traversal | Hard]

What is the output?

```
s = "abcde"
result = ""
for i in range(len(s)):
result += s[i] * i
print(result)
```

(a) abbcccddddd
(b) aabbbcccc
(c) abcde
(d) bccdddeeee

Q46. [operations | Hard]

What is printed?

```
a = [1, 2, 3]
b = a * 2
b[0] = 99
print(a[0], b[0])
```

(a) 1 1
(b) 99 99
(c) 1 99
(d) 99 1

Q47. [list parameters and nested list | Hard]

What is the output?

```
def modify(lst):
lst = lst + [4]
lst.append(5)
x = [1, 2, 3]
modify(x)
print(x)
```

(a) [1, 2, 3, 5]
(b) [1, 2, 3, 4]
(c) [1, 2, 3]
(d) [1, 2, 3, 4, 5]

Q48. [slices | Hard]

What is the result?

```
lst = [0, 1, 2, 3, 4, 5]
lst[1:4] = [10, 20]
print(lst)
```

(a) [0, 10, 20, 3, 4, 5]
(b) [0, 10, 20, 4, 5]
(c) [0, 10, 20, 2, 3, 4, 5]
(d) [10, 20, 3, 4, 5]

Q49. [deletion | Hard]

What is the output?

```
lst = [10, 20, 30, 40, 50]
del lst[1:4:2]
print(lst)
```

(a) `[10, 30, 40, 50]`
(b) `[10, 30, 50]`
(c) `[10, 40, 50]`
(d) `[20, 40]`

Q50. [list and for loops | Hard]

What does this print?

```
matrix = [[1, 2], [3, 4], [5, 6]]
total = 0
for row in matrix:
total += row[-1]
print(total)
```

(a) `9`
(b) `12`
(c) `21`
(d) `6`

Q51. [membership | Hard]

What is the output?

```
nested = [[1, 2], [3, 4]]
print(3 in nested, [3, 4] in nested)
```

(a) `False True`
(b) `True False`
(c) `False False`
(d) `True True`

Q52. [accessing elements | Hard]

What is printed?

```
grid = [[0]*3]*3
grid[0][0] = 1
print(grid)
```

(a) `[[1, 0, 0], [0, 0, 0], [1, 0, 0]]`
(b) `[[1, 0, 0], [1, 0, 0], [1, 0, 0]]`
(c) `[[1, 1, 1], [0, 0, 0], [0, 0, 0]]`
(d) `[[1, 0, 0], [0, 0, 0], [0, 0, 0]]`

Q53. [tuples | Hard]

Which of the following creates a tuple containing a single integer element?

(a) `t = tuple(5)`
(b) `t = (5,)`
(c) `t = (5)`
(d) `t = 5`

Q54. [operations on tuples | Hard]

What is the output?

```
t = (1, [2, 3], 4)
t[1].append(5)
print(t)
```

(a) `(1, [2, 3, 5], 4)`
(b) `(1, [2, 3], 4)`
(c) `TypeError raised`
(d) `(1, [2, 3], 4, 5)`

Q55. [creating dictionary | Hard]

What does the following produce?

```
d = dict(zip("abc", [1, 2, 3, 4]))
print(d)
```

(a) {'a': 1, 'b': 2, 'c': 3, None: 4}
(b) ValueError raised
(c) {'a': 1, 'b': 2, 'c': 3, 'd': 4}
(d) {'a': 1, 'b': 2, 'c': 3}

Q56. [adding-modifying-retrieving dictionary values | Hard]

What is printed?

```
d = {'x': 1}
print(d.get('y', 0) + d.setdefault('y', 5))
print(d)
```

(a) 5 then {'x': 1}
(b) 5 then {'x': 1, 'y': 5}
(c) 0 then {'x': 1}
(d) 10 then {'x': 1, 'y': 5}

Q57. [deleting items | Hard]

What is the output?

```
d = {'a': 1, 'b': 2, 'c': 3}
val = d.pop('b')
res = d.pop('z', -1)
print(val, res, len(d))
```

(a) 2 -1 2
(b) 2 None 2
(c) 2 -1 3
(d) None -1 2

Q58. [dictionary methods | Hard]

What does this print?

```
d = {'a': 1, 'b': 2}
d.update({'b': 20, 'c': 30})
print(sum(d.values()))
```

(a) 51
(b) 53
(c) 33
(d) 23

Q59. [operations on dictionary | Hard]

What is the output?

```
d = {1: 'a', 2: 'b'}
for k in list(d.keys()):
d[k*10] = d[k]
print(len(d))
```

(a) 4
(b) 6
(c) RuntimeError raised
(d) 2

Q60. [length | Hard]

What is printed?

```
s = "a\{}tb\{}nc"
print(len(s))
```

(a) 5
(b) 7

(c) `3`
(d) `6`

3.2 Section B: Comprehensive Subjective Problem with Detailed Analytical Rubric

Problem 3 **[Subjective Analytical Problem — 10 Marks]**

1 Design an industrial-grade string manipulation and nested data structure algorithm for inventory and text parsing:

(a) Write a function `word_frequency_matrix(corpus)` that cleans text (removing punctuation, case normalization), counts token frequency, and returns a sorted descending dictionary.

(b) Write a function `merge_inventories(warehouse_a, warehouse_b)` that merges two nested dictionaries containing item codes, quantities, and unit prices, computing the aggregated inventory value and updating total quantities.

Step-by-Step Analytical & Algorithmic Solution:

Python Solution:

```
import string

def word_frequency_matrix(corpus):
  clean_tokens = corpus.lower().translate(str.maketrans("", "", string.punctuation)).split()
  freq = {}
  for tok in clean_tokens:
    freq[tok] = freq.get(tok, 0) + 1
  sorted_freq = dict(sorted(freq.items(), key=lambda item:  item[1], reverse=True))
  return sorted_freq

def merge_inventories(w1, w2):
  merged = {}
  all_keys = set(w1.keys()) | set(w2.keys())
  for k in all_keys:
    q1 = w1.get(k, {}).get("qty", 0)
    q2 = w2.get(k, {}).get("qty", 0)
    p1 = w1.get(k, {}).get("price", 0.0)
    p2 = w2.get(k, {}).get("price", 0.0)
    price = max(p1, p2)
    merged[k] = {"qty":  q1 + q2, "price":  price, "total_val":  (q1 + q2) * price}
  return merged
```

Complexity Analysis: Dictionary lookups and insertions operate in average $\mathcal{O}(1)$ time. Set union executes in $\mathcal{O}(|w_1| + |w_2|)$, ensuring complete $\mathcal{O}(N \log N)$ complexity dominated by dictionary sorting.

- **Algorithmic Formulation (3 Marks):** Correct problem decomposition, function signatures, and parameter documentation.
- **Syntactic Rigor & Implementation (4 Marks):** Valid Python syntax, proper error handling, defensive input validation, and avoidance of anti-patterns.
- **Complexity & Edge Analysis (3 Marks):** Accurate time and space asymptotic analysis $\mathcal{O}(n)$ and testing edge conditions (null inputs, zero division, empty collections).

Chapter 4

Unit 4: Array Operations using NumPy

4.1 Section A: Multiple Choice Practice Questions (60 Curated Problems)

This section contains 60 rigorous multiple-choice questions extracted from university course examination assessments. Review each question, test edge-case syntax, and verify code execution mechanics.

Q1. [arrays vs lists | Easy]

Which Python library provides the `ndarray` object used for numerical arrays?

(a) Matplotlib
(b) NumPy
(c) SciPy
(d) Pandas

Q2. [arrays vs lists | Easy]

What is a key difference between a NumPy array and a Python list?

(a) A NumPy array cannot be indexed
(b) A list uses less memory than an array
(c) A NumPy array stores elements of a single data type
(d) A list can only hold numbers

Q3. [arrays vs lists | Easy]

Which function is used to create a NumPy array from a Python list?

(a) `np.list()`
(b) `np.make()`
(c) `np.create()`
(d) `np.array()`

Q4. [arrays vs lists | Easy]

Compared to Python lists, NumPy arrays are generally preferred for numerical computation because they are:

(a) Limited to one element only
(b) Faster and more memory efficient
(c) Unable to store integers
(d) Slower but easier to read

Q5. [data types | Easy]

Which attribute is used to check the data type of elements in a NumPy array?

(a) `.datatype`
(b) `.dtype`
(c) `.type`
(d) `.kind`

Q6. [data types | Easy]

What does the NumPy data type `int64` represent?

(a) A 64-character string
(b) A 64-element array
(c) A 64-bit floating point number
(d) A 64-bit integer

Q7. [data types | Easy]

Which method changes the data type of an existing NumPy array?

(a) `.astype()`
(b) `.convert()`
(c) `.retype()`
(d) `.setdtype()`

Q8. [data types | Easy]

If an array is created as `np.array([1.0, 2.0, 3.0])`, its default data type will be:

(a) `bool`
(b) `str`
(c) `float64`
(d) `int32`

Q9. [array operations | Easy]

Given `a = np.array([1, 2, 3])`, what is the result of `a + 2`?

(a) `[3, 4, 5]`
(b) `[1, 2, 3, 2]`
(c) `[2, 4, 6]`
(d) `[3, 2, 3]`

Q10. [array operations | Easy]

Which attribute returns the shape (dimensions) of a NumPy array?

(a) `.size`
(b) `.shape`
(c) `.dim`
(d) `.length`

Q11. [array operations | Easy]

Given `a = np.array([1, 2, 3])` and `b = np.array([4, 5, 6])`, what is `a * b`?

(a) `[4, 5, 6]`
(b) `[1, 2, 3]`
(c) `[4, 10, 18]`
(d) `[5, 7, 9]`

Q12. [array operations | Easy]

Which function reshapes an array without changing its data?

(a) `restack()`
(b) `resize()`
(c) `reform()`
(d) `reshape()`

Q13. [array operations | Easy]

What does `np.arange(0, 5)` produce?

(a) `[0, 1, 2, 3, 4]`
(b) `[0, 1, 2, 3, 4, 5]`
(c) `[1, 2, 3, 4, 5]`
(d) `[0, 5]`

Q14. [statistical functions | Easy]

Which NumPy function computes the average of array elements?

(a) `np.mean()`
(b) `np.middle()`
(c) `np.center()`
(d) `np.avg()`

Q15. [statistical functions | Easy]
What does `np.max()` return for a NumPy array?
(a) The number of elements
(b) The smallest element
(c) The sum of elements
(d) The largest element

Q16. [statistical functions | Easy]
Which function computes the standard deviation of a NumPy array?
(a) `np.dev()`
(b) `np.std()`
(c) `np.var()`
(d) `np.spread()`

Q17. [statistical functions | Easy]
Given `a = np.array([2, 4, 6])`, what does `np.sum(a)` return?
(a) `3`
(b) `12`
(c) `6`
(d) `4`

Q18. [broadcasting | Easy]
What does broadcasting in NumPy allow you to do?
(a) Sort arrays automatically
(b) Print arrays to screen
(c) Send data over a network
(d) Operate on arrays of different shapes

Q19. [broadcasting | Easy]
When you add a scalar to a NumPy array, broadcasting causes the scalar to be:
(a) Ignored entirely
(b) Applied to every element
(c) Added only to the first element
(d) Converted to a string

Q20. [broadcasting | Easy]
For broadcasting to work between two arrays, their dimensions must be:
(a) Prime numbers
(b) Compatible or equal to 1
(c) Both greater than 10
(d) Exactly identical always

Q21. [arrays vs lists | Medium]
Given `import numpy as np`, what is the output of the following code? python lst = [1, 2, 3] arr = np.array([1, 2, 3]) print(lst * 2, arr * 2)
(a) `[2, 4, 6] [2 4 6]`
(b) `[2, 4, 6] [1 2 3 1 2 3]`
(c) `[1, 2, 3, 1, 2, 3] [1 2 3 1 2 3]`
(d) `[1, 2, 3, 1, 2, 3] [2 4 6]`

Q22. [arrays vs lists | Medium]
Which statement best explains why NumPy arrays are generally faster than Python lists for numerical computa-

tion?

(a) Arrays are interpreted line by line while lists are compiled
(b) Arrays store each element as a separate Python object with pointers
(c) Arrays automatically use multiple CPU cores for every operation
(d) Arrays store homogeneous data in contiguous memory and use vectorized C operations

Q23. [data types | Medium]

What will be the `dtype` of the array created below? python arr = np.array([1, 2, 3.0, 4])

(a) `float64`
(b) `int64`
(c) `object`
(d) `complex128`

Q24. [data types | Medium]

What is the output of the following code? python arr = np.array([1, 2, 3], dtype=np.int8) arr[0] = 300 print(arr[0])

(a) Raises an OverflowError
(b) `300`
(c) `127`
(d) `44`

Q25. [data types | Medium]

Which code correctly converts a float array `a` into an integer array without changing the original?

(a) `b = a.dtype(np.int32)`
(b) `b = a.convert('int32')`
(c) `b = int(a)`
(d) `b = a.astype(np.int32)`

Q26. [array operations | Medium]

Given `arr = np.arange(12).reshape(3, 4)`, what does `arr[:, 1]` return?

(a) `[1 2 3 4]`
(b) `[1 5 9]`
(c) `[0 1 2 3]`
(d) `[4 5 6 7]`

Q27. [array operations | Medium]

What is the result of the following code? python a = np.array([[1, 2], [3, 4]]) print(a.sum(axis=0))

(a) `[4 6]`
(b) `[3 7]`
(c) `[6 4]`
(d) `10`

Q28. [array operations | Medium]

Given `arr = np.array([10, 20, 30, 40, 50])`, what does `arr[::-2]` return?

(a) `[10 30 50]`
(b) `[40 20]`
(c) `[50 30 10]`
(d) `[50 40 30 20 10]`

Q29. [array operations | Medium]

What is the difference between `np.dot(a, b)` and `a * b` for two 2D arrays of the same shape?

(a) `np.dot` performs element-wise multiplication while `*` performs matrix multiplication
(b) Both perform matrix multiplication but `*` is faster
(c) Both perform element-wise multiplication with different rounding
(d) `np.dot` performs matrix multiplication while `*` performs element-wise multiplication

Q30. [array operations | Medium]

What is the output of the following code? python arr = np.array([1, 2, 3, 4, 5]) print(arr[arr % 2 == 0])

(a) `[True False True False True]`
(b) `[1 3 5]`
(c) `[2 4]`
(d) `[0 2 0 4 0]`

Q31. [statistical functions | Medium]

Given `arr = np.array([2, 4, 6, 8])`, what does `np.std(arr)` compute and its approximate value?

(a) The variance, 5.0
(b) Sample standard deviation, ≈
(c) Population standard deviation, ≈
(d) The range, 6.0

Q32. [statistical functions | Medium]

For `a = np.array([[1, 2], [3, 4]])`, what does `np.mean(a, axis=1)` return?

(a) `[1.5 3.5]`
(b) `2.5`
(c) `[2.  3.]`
(d) `[4.  6.]`

Q33. [statistical functions | Medium]

Which function returns the *index* of the maximum value in an array rather than the value itself?

(a) `np.maximum`
(b) `np.argmax`
(c) `np.max`
(d) `np.amax`

Q34. [statistical functions | Medium]

What is the output of the following code? python arr = np.array([3, 1, 4, 1, 5, 9, 2]) print(np.median(arr))

(a) `4.0`
(b) `3.5`
(c) `1.0`
(d) `3.0`

Q35. [statistical functions | Medium]

Given `arr = np.array([10, 20, 30, 40])`, what does `np.cumsum(arr)` return?

(a) `[10 20 30 40]`
(b) `100`
(c) `[10 30 60 100]`
(d) `[100 90 70 40]`

Q36. [broadcasting | Medium]

What is the result of the following code? python a = np.array([[1], [2], [3]]) b = np.array([10, 20, 30]) print((a + b).shape)

(a) `(3, 3)`
(b) Raises a ValueError
(c) `(3,)`
(d) `(3, 1)`

Q37. [broadcasting | Medium]

Which pair of array shapes CANNOT be broadcast together?

(a) `(5, 1)` and `(1, 6)`
(b) `(3, 4)` and `(4,)`
(c) `(4, 3)` and `(2,)`
(d) `(2, 3)` and `(1, 3)`

Q38. [broadcasting | Medium]

What is the output of the following code? python a = np.array([1, 2, 3]) print(a + 5)

(a) `[1 2 3 5]`
(b) `[5 5 5]`
(c) `[6 7 8]`
(d) Raises a ValueError

Q39. [broadcasting | Medium]

Given `a` of shape `(2, 3)` and `b` of shape `(3,)`, what is the shape of `a * b`?

(a) Raises a ValueError
(b) `(2,)`
(c) `(3,)`
(d) `(2, 3)`

Q40. [array operations | Medium]

What is the output of the following code? python arr = np.array([1, 2, 3]) arr2 = arr arr2[0] = 99 print(arr[0])

(a) `0`
(b) Raises an error
(c) `1`
(d) `99`

Q41. [arrays vs lists | Hard]

Consider the following code:

```
 import numpy as np
lst = [1, 2, 3]
arr = np.array([1, 2, 3])
print(lst * 2)
print(arr * 2)
```

What is the output?

(a) `[1, 2, 3, 1, 2, 3]` and `[1 2 3 1 2 3]`
(b) `[2, 4, 6]` and `[1 2 3 1 2 3]`
(c) `[1, 2, 3, 1, 2, 3]` and `[2 4 6]`
(d) `[2, 4, 6]` and `[2 4 6]`

Q42. [arrays vs lists | Hard]

A NumPy array stores 1 0 6 integers while an equivalent Python list stores the same values. Why does the NumPy array typically consume far less memory?

(a) It stores only unique values and references duplicates
(b) It offloads storage to disk and keeps only pointers in RAM
(c) It compresses the integers using run-length encoding automatically
(d) It stores raw values in a contiguous, fixed-type buffer without per-element object overhead

Q43. [data types | Hard]

What does the following code print?

```
 import numpy as np
arr = np.array([1, 2, 3], dtype=np.int8)
arr[0] = 130
print(arr[0])
```

(a) `-126`
(b) `130`
(c) `127`
(d) `2`

Q44. [data types | Hard]

Given `arr = np.array([1, 2, 3])`, which operation returns a new array of type `float64` without modifying `arr`?

(a) `arr.view(np.float64)`

(b) `arr.astype(np.float64)`
(c) `arr.dtype = np.float64`
(d) `arr.reshape(np.float64)`

Q45. [data types | Hard]
What is the resulting dtype of `np.array([True, 2, 3.5])`?
(a) `int64`
(b) `float64`
(c) `object`
(d) `bool`

Q46. [array operations | Hard]
What does the following print?

```
import numpy as np
a = np.arange(6).reshape(2, 3)
b = a.T
b[0, 0] = 99
print(a[0, 0])
```

(a) `0`
(b) `99`
(c) `5`
(d) Raises an error

Q47. [array operations | Hard]
Given `a = np.array([10, 20, 30, 40])`, what does `a[::-1][1:3]` return?
(a) `array([30, 20])`
(b) `array([20, 30])`
(c) `array([40, 30])`
(d) `array([30, 40])`

Q48. [array operations | Hard]
Consider:

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
mask = a % 2 == 0
a[mask] = 0
print(a)
```

What is the output?
(a) `[0 2 0 4 0]`
(b) `[0 0 0 0 0]`
(c) `[1 2 3 4 5]`
(d) `[1 0 3 0 5]`

Q49. [array operations | Hard]
What is the result of `np.arange(12).reshape(3, 4)[:, 1::2]`?
(a) `[[2 3], [6 7], [10 11]]`
(b) `[[1 3], [5 7], [9 11]]`
(c) `[[0 2], [4 6], [8 10]]`
(d) `[[1 2], [5 6], [9 10]]`

Q50. [array operations | Hard]
Which statement about `np.reshape(a, (-1, 3))` is correct when `a` has 12 elements?
(a) It raises an error because `-1` is not a valid dimension
(b) The `-1` is treated as 0
(c) The `-1` is inferred as 4
(d) The `-1` reverses the array before reshaping

Q51. [statistical functions | Hard]

For `a = np.array([[1, 2], [3, 4]])`, what does `a.mean(axis=0)` return?

(a) `array([1.5, 2.5, 3.5])`
(b) `array([1.5, 3.5])`
(c) `array([2., 3.])`
(d) `2.5`

Q52. [statistical functions | Hard]

What does `np.std(np.array([2, 4, 4, 4, 5, 5, 7, 9]))` return (population standard deviation)?

(a) `4.0`
(b) `2.14`
(c) `2.0`
(d) `5.0`

Q53. [statistical functions | Hard]

Given an array containing a `np.nan`, which function ignores the NaN when computing the mean?

(a) `np.nanmean`
(b) `np.mean`
(c) `np.mean` with `axis=None`
(d) `np.average`

Q54. [statistical functions | Hard]

What is the difference between `a.var()` and `a.var(ddof=1)` for a sample array?

(a) `ddof=1` divides by N +
(b) `ddof=1` divides by N -
(c) They are identical for all arrays
(d) `ddof=1` computes standard deviation instead of variance

Q55. [statistical functions | Hard]

For `a = np.array([[1, 5], [3, 2]])`, what does `np.argmax(a)` return?

(a) `3`
(b) `5`
(c) `(0, 1)`
(d) `1`

Q56. [broadcasting | Hard]

Given `a` of shape `(3, 1)` and `b` of shape `(1, 4)`, what is the shape of `a + b`?

(a) Raises a broadcasting error
(b) `(3, 1)`
(c) `(3, 4)`
(d) `(1, 4)`

Q57. [broadcasting | Hard]

Which pair of shapes CANNOT be broadcast together?

(a) `(5, 3, 4)` and `(3, 4)`
(b) `(3, 1)` and `(1, 4)`
(c) `(3, 4)` and `(2, 4)`
(d) `(3, 4)` and `(4,)`

Q58. [broadcasting | Hard]

What does the following produce?

```
 import numpy as np
a = np.array([[1], [2], [3]])
b = np.array([10, 20, 30])
print((a + b).shape)
```

(a) `(1, 3)`

(b) `(9,)`
(c) `(3, 3)`
(d) `(3, 1)`

Q59. [broadcasting | Hard]

To subtract the per-column mean from each element of a `(100, 5)` array `X`, which expression works via broadcasting?

(a) `X - X.mean()`
(b) `X - X.mean(axis=1).reshape(-1, 1)` giving column-mean subtraction
(c) `X - X.mean(axis=0)`
(d) `X - X.mean(axis=1)`

Q60. [broadcasting | Hard]

What is the result of the following?

```
 import numpy as np
a = np.array([1, 2, 3])
b = np.array([[0], [1], [2]])
print(a * b)
```

(a) `[[0 2 6]]`
(b) `[[1 2 3], [1 2 3], [1 2 3]]`
(c) Raises a broadcasting error
(d) `[[0 0 0], [1 2 3], [2 4 6]]`

4.2 Section B: Comprehensive Subjective Problem with Detailed Analytical Rubric

Problem 4 **[Subjective Analytical Problem — 10 Marks]**

1 Demonstrate NumPy vectorization, multi-dimensional array manipulation, and statistical broadcasting:

(a) Given a 1000×10 sensor dataset, normalize each feature column to zero-mean and unit variance (z-score standardization) using vectorized operations without any explicit Python `for` loops.

(b) Implement min-max feature scaling $[0, 1]$ utilizing column-wise broadcasting.

(c) Compute pairwise Euclidean distances between two matrices $X \in \mathbb{R}^{m \times d}$ and $Y \in \mathbb{R}^{n \times d}$ using NumPy broadcasting.

Step-by-Step Analytical & Algorithmic Solution:

Vectorized NumPy Solution:

```
import numpy as np

def standardize_matrix(X):
  mean = np.mean(X, axis=0)   # shape:  (10,)
  std = np.std(X, axis=0)     # shape:  (10,)
  std_safe = np.where(std == 0, 1.0, std)
  return (X - mean) / std_safe   # Broadcasts (1000, 10) - (10,)

def min_max_scale(X):
  x_min = np.min(X, axis=0)
  x_max = np.max(X, axis=0)
  diff = np.where(x_max == x_min, 1.0, x_max - x_min)
  return (X - x_min) / diff

def pairwise_euclidean(X, Y):
  # X: (m, 1, d), Y: (1, n, d) -> diff:  (m, n, d)
  diff = X[:, np.newaxis, :]  - Y[np.newaxis, :, :]
  return np.sqrt(np.sum(diff ** 2, axis=-1))
```

Broadcasting Invariants: Alignment occurs along trailing dimensions. Operating in C-contiguous memory blocks yields 50x–100x acceleration over pure Python iterations due to SIMD vector pipelines and cache spatial locality.

- **Algorithmic Formulation (3 Marks):** Correct problem decomposition, function signatures, and parameter documentation.
- **Syntactic Rigor & Implementation (4 Marks):** Valid Python syntax, proper error handling, defensive input validation, and avoidance of anti-patterns.
- **Complexity & Edge Analysis (3 Marks):** Accurate time and space asymptotic analysis $\mathcal{O}(n)$ and testing edge conditions (null inputs, zero division, empty collections).

Chapter 5

Unit 5: Handling Data with Pandas and Data Visualisation with Matplotlib

5.1 Section A: Multiple Choice Practice Questions (60 Curated Problems)

This section contains 60 rigorous multiple-choice questions extracted from university course examination assessments. Review each question, test edge-case syntax, and verify code execution mechanics.

Q1. [introduction to pandas | Easy]

What is Pandas primarily used for in Python?

(a) Managing operating system processes
(b) Data manipulation and analysis
(c) Compiling C++ code
(d) Building 3D games

Q2. [introduction to pandas | Easy]

Which is the conventional way to import the Pandas library?

(a) `include pandas as pd`
(b) `import pd as pandas`
(c) `import pandas as pd`
(d) `import pandas.pd`

Q3. [introduction to pandas | Easy]

Pandas is built on top of which numerical computing library?

(a) Matplotlib
(b) TensorFlow
(c) SciPy
(d) NumPy

Q4. [series | Easy]

A Pandas Series is best described as a:

(a) Two-dimensional labeled table
(b) One-dimensional labeled array
(c) Three-dimensional data cube
(d) Scalar single value

Q5. [series | Easy]

Which function is used to create a Pandas Series?

(a) `pd.DataFrame()`
(b) `pd.array()`
(c) `pd.Series()`

(d) `pd.series()`

Q6. [series | Easy]
What is the default index for a Series created from a list of 4 elements?
(a) Integers 1, 2, 3, 4
(b) Integers 0, 1, 2, 3
(c) Letters a, b, c, d
(d) No index is assigned

Q7. [dataframe | Easy]
A Pandas DataFrame is a:
(a) One-dimensional labeled array
(b) Single scalar value
(c) Plain Python dictionary
(d) Two-dimensional labeled data structure

Q8. [dataframe | Easy]
Which method displays the first 5 rows of a DataFrame `df` by default?
(a) `df.first()`
(b) `df.head()`
(c) `df.top()`
(d) `df.start()`

Q9. [dataframe | Easy]
Which attribute gives the number of rows and columns of a DataFrame as a tuple?
(a) `df.dimensions`
(b) `df.length`
(c) `df.size`
(d) `df.shape`

Q10. [working with csv files | Easy]
Which Pandas function reads data from a CSV file into a DataFrame?
(a) `pd.read_csv()`
(b) `pd.import_csv()`
(c) `pd.load_csv()`
(d) `pd.open_csv()`

Q11. [working with csv files | Easy]
Which method writes a DataFrame `df` to a CSV file?
(a) `df.export_csv()`
(b) `df.write_csv()`
(c) `df.save_csv()`
(d) `df.to_csv()`

Q12. [working with csv files | Easy]
What does CSV stand for?
(a) Character-Set Values
(b) Common Storage Version
(c) Column-Sorted Values
(d) Comma-Separated Values

Q13. [operations using dataframes | Easy]
Which method provides summary statistics like mean, min, and max for numeric columns?
(a) `df.stats()`
(b) `df.summary()`
(c) `df.info()`
(d) `df.describe()`

Q14. [operations using dataframes | Easy]

How do you select a single column named `age` from a DataFrame `df`?

(a) `df['age']`
(b) `df.column('age')`
(c) `df.get('age', all=True)`
(d) `df->age`

Q15. [operations using dataframes | Easy]

Which method is used to remove rows containing missing (NaN) values?

(a) `df.deletena()`
(b) `df.removeNaN()`
(c) `df.clearna()`
(d) `df.dropna()`

Q16. [line plots | Easy]

Which Matplotlib function is used to create a line plot ?

(a) `plt.line()`
(b) `plt.graph()`
(c) `plt.plot()`
(d) `plt.draw()`

Q17. [multiple subplots in one figure | Easy]

Which function creates a figure with multiple subplots at once?

(a) `plt.subplots()`
(b) `plt.multiplot()`
(c) `plt.grid()`
(d) `plt.figures()`

Q18. [histograms | Easy]

A histogram is mainly used to show the:

(a) Trend over time only
(b) Distribution of a numeric variable
(c) Proportion of a whole
(d) Relationship between two categories

Q19. [bar charts | Easy]

Which Matplotlib function creates a vertical bar chart ?

(a) `plt.bar()`
(b) `plt.hist()`
(c) `plt.barh()`
(d) `plt.column()`

Q20. [pie charts | Easy]

A pie chart is best suited for showing:

(a) Frequency distribution of bins
(b) Proportions of a whole
(c) Correlation between variables
(d) Trends over time

Q21. [introduction to pandas | Medium]

You import pandas using `import pandas as pd`. Which statement correctly describes why pandas is preferred over plain Python lists for tabular data analysis?

(a) It replaces the need for the NumPy library entirely
(b) It compiles Python code into C for faster loops
(c) It provides labeled, aligned data structures with vectorized operations
(d) It stores data only as JSON internally for speed

Q22. [series | Medium]

Given `s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])`, what does `s['b']` return?

(a) `10`
(b) `['b', 20]`
(c) `30`
(d) `20`

Q23. [series | Medium]

What is the result of `pd.Series([1, 2, 3]) + pd.Series([10, 20], index=[0, 2])`?

(a) It raises a length mismatch error
(b) All values are added position by position ignoring the index
(c) It concatenates into a Series of length 5
(d) Values at index 1 become `NaN` due to alignment

Q24. [dataframe | Medium]

For a DataFrame `df`, which expression selects only the rows where the column `age` is greater than 30?

(a) `df[df.age.filter(> 30)]`
(b) `df[df['age'] > 30]`
(c) `df.loc['age' > 30]`
(d) `df['age' > 30]`

Q25. [dataframe | Medium]

What does `df.shape` return for a DataFrame with 100 rows and 5 columns?

(a) `(5, 100)`
(b) `[100, 5]`
(c) `(100, 5)`
(d) `500`

Q26. [dataframe | Medium]

Which method would you use to view the first 5 rows of a DataFrame `df`?

(a) `df.first(5)`
(b) `df.top()`
(c) `df.begin()`
(d) `df.head()`

Q27. [working with csv files | Medium]

You have a CSV file where columns are separated by semicolons (;). Which call reads it correctly?

(a) `pd.read_csv('data.csv', delim=';')`
(b) `pd.read_csv('data.csv', split=';')`
(c) `pd.read_csv('data.csv', sep=';')`
(d) `pd.load_csv('data.csv', sep=';')`

Q28. [working with csv files | Medium]

You want to save a DataFrame `df` to a CSV file without writing the index column. Which call is correct?

(a) `df.write_csv('out.csv', index=None)`
(b) `df.to_csv('out.csv', index=False)`
(c) `df.save_csv('out.csv', index=0)`
(d) `df.to_csv('out.csv', header=False)`

Q29. [working with csv files | Medium]

When reading a CSV that has no header row, which argument ensures pandas does not treat the first data row as column names?

(a) `skiprows=1`
(b) `header=None`
(c) `names=True`
(d) `header=0`

Q30. [operations using dataframes | Medium]

Given a DataFrame `df` with a numeric column `sales`, which expression computes the average of that column?

(a) `df['sales'].mean()`
(b) `df.mean('sales')`
(c) `mean(df['sales'])`
(d) `df['sales'].average()`

Q31. [operations using dataframes | Medium]

What does `df.groupby('city')['sales'].sum()` produce?

(a) A count of rows per city
(b) Total sales for each unique city
(c) The sales sorted alphabetically by city
(d) The overall sum of all sales values

Q32. [operations using dataframes | Medium]

Which method returns the count of missing (`NaN`) values in each column of a DataFrame `df`?

(a) `df.isnull().sum()`
(b) `df.isnull().count()`
(c) `df.dropna().sum()`
(d) `df.missing().sum()`

Q33. [operations using dataframes | Medium]

You want to create a new column `total` equal to `price` multiplied by `quantity`. Which statement is correct?

(a) `df.total = multiply(price, quantity)`
(b) `df['total'] = df['price'].mul()`
(c) `df.add('total', price * quantity)`
(d) `df['total'] = df['price'] * df['quantity']`

Q34. [line plots | Medium]

Using matplotlib, which pair of calls plots `y` against `x` as a line and then displays it?

(a) `plt.lineplot(x, y); plt.show()`
(b) `plt.line(x, y); plt.draw()`
(c) `plt.plot(y, x); plt.render()`
(d) `plt.plot(x, y); plt.show()`

Q35. [line plots | Medium]

In `plt.plot(x, y, 'r-')`, what does the format string `'r-'` specify?

(a) A rectangular line style
(b) A red dashed line
(c) Two red solid lines
(d) A round dotted marker

Q36. [multiple subplots in one figure | Medium]

Which call creates a figure with a 2-row by 2-column grid of subplots?

(a) `fig, ax = plt.figure(2, 2)`
(b) `fig, ax = plt.subplot(2, 2)`
(c) `fig, ax = plt.grid(2, 2)`
(d) `fig, ax = plt.subplots(2, 2)`

Q37. [multiple subplots in one figure | Medium]

After `fig, axes = plt.subplots(2, 2)`, how do you draw on the subplot in the bottom-right corner?

(a) `axes['bottom-right'].plot(...)`
(b) `axes(1, 1).plot(...)`
(c) `axes[2, 2].plot(...)`
(d) `axes[1, 1].plot(...)`

Q38. [histograms | Medium]

Which parameter of `plt.hist(data, bins=20)` controls the number of intervals the data range is divided into?

(a) `width`
(b) `bins`
(c) `range`
(d) `intervals`

Q39. [histograms | Medium]

A histogram is most appropriate for visualizing which of the following?

(a) The proportion of parts in a whole
(b) The relationship between two categorical variables
(c) The frequency distribution of a single continuous variable
(d) The exact value of each individual record

Q40. [bar charts | Medium]

You have categories in a list `cats` and their counts in `vals`. Which call creates a vertical bar chart?

(a) `plt.plot(cats, vals, 'bar')`
(b) `plt.bar(cats, vals)`
(c) `plt.hist(cats, vals)`
(d) `plt.barh(cats, vals)`

Q41. [series | Hard]

Given `s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])`, what does `s[0:2]` return compared to `s['a':'b']`?

(a) `s[0:2]` raises a `KeyError` since integer indexing is disabled when labels exist
(b) Both return exactly the same two elements because slicing is always position-based
(c) `s['a':'b']` excludes 'b' because all Python slicing is exclusive of the endpoint
(d) `s[0:2]` returns elements at positions 0 and 1; `s['a':'b']` returns elements 'a' and 'b' (label slicing is inclusive of endpoint)

Q42. [dataframe | Hard]

For a DataFrame `df`, which statement correctly distinguishes `df.loc[]` from `df.iloc[]` when the index is `[2, 0, 1]`?

(a) Both select the same row because index labels are ignored
(b) `df.loc[0]` raises an error because the index is not sorted
(c) `df.iloc[0]` selects the row labeled 0 and `df.loc[0]` selects the first physical row
(d) `df.loc[0]` selects the row with index label 0; `df.iloc[0]` selects the first physical row (label 2)

Q43. [operations using dataframes | Hard]

Given `df1` with index `['a','b']` and `df2` with index `['b','c']`, what is the result of `df1 + df2` for overlapping and non-overlapping labels?

(a) All rows are summed positionally regardless of labels
(b) Rows 'a' and 'c' become `NaN` (no alignment match); row 'b' holds the element-wise sum
(c) The operation raises a `ValueError` due to mismatched indices
(d) Only row 'b' survives; rows 'a' and 'c' are dropped from the result

Q44. [working with csv files | Hard]

A CSV has a column of integers but some cells are empty. After `pd.read_csv('data.csv')`, what dtype does that column typically have and why?

(a) `int64`, because pandas fills missing integers with 0 automatically
(b) `Int64`, because pandas always uses nullable integer types by default
(c) `float64`, because `NaN` (a float) forces the entire integer column to be upcast to float
(d) `object`, because any missing value converts all numbers to strings

Q45. [operations using dataframes | Hard]

For a numeric DataFrame `df`, how do `df.apply(np.sum, axis=0)` and `df.apply(np.sum, axis=1)` differ?

(a) Both return a single scalar equal to the total sum of all elements
(b) `axis=0` returns per-row sums; `axis=1` returns per-column sums
(c) `axis=0` returns one sum per column (collapsing rows); `axis=1` returns one sum per row (collapsing columns)

(d) `axis=1` raises an error because `np.sum` only works column-wise

Q46. [dataframe | Hard]

What is the key behavioral difference between `df.dropna(how='all')` and `df.dropna(how='any')` (default)?

(a) `how='all'` fills `NaN` with the mean instead of dropping
(b) Both drop identical rows; the parameter only changes performance
(c) `how='all'` drops rows with any `NaN`; `how='any'` drops only fully-`NaN` rows
(d) `how='all'` drops a row only if every value is `NaN`; `how='any'` drops a row if at least one value is `NaN`

Q47. [line plots | Hard]

When plotting `plt.plot(x, y1); plt.plot(x, y2)` on the same axes without specifying colors, what happens?

(a) The second `plot` call overwrites and erases the first line
(b) Both lines are drawn in the same color, making them indistinguishable
(c) A `ValueError` is raised because colors must be specified explicitly
(d) Matplotlib automatically assigns different colors from the default color cycle to each line

Q48. [multiple subplots in one figure | Hard]

Using `fig, ax = plt.subplots(2, 2)`, how should you correctly access the subplot in the bottom-right position?

(a) `ax[2][2]`, using 1-based row and column numbers
(b) `ax[3]`, because subplots are stored in a flat 1D list
(c) `ax[1, 1]`, because `ax` is a 2D NumPy array of Axes indexed by [row, column]
(d) `ax.bottom_right`, using the named-position attribute

Q49. [histograms | Hard]

For `plt.hist(data, bins=10)` on data ranging from 0 to 100, what determines each bin's width and count?

(a) Width is fixed at 1 unit regardless of the data range
(b) Width is the range divided by bins (10 units each); count is the number of data points falling in each interval
(c) The count is the cumulative total of all points up to that bin's upper edge
(d) Each bin always contains exactly the same number of data points (equal-frequency binning)

Q50. [bar charts | Hard]

What is the fundamental difference between `plt.bar()` and `plt.hist()`?

(a) `bar()` can only produce horizontal bars while `hist()` produces vertical ones
(b) `bar()` bins continuous data automatically while `hist()` requires pre-counted categories
(c) `bar()` plots one bar per categorical/discrete value you supply; `hist()` bins continuous data and plots frequencies of those bins
(d) Both are identical; `hist` is just an alias for `bar`

Q51. [pie charts | Hard]

In `plt.pie(sizes, autopct='%1.1f%%')`, what does `autopct` control and how are wedge sizes determined?

(a) `autopct` controls wedge color; sizes are determined by insertion order only
(b) `autopct` sets the number of wedges; wedge sizes are always equal
(c) `autopct` rotates the chart; wedge sizes come from raw values without normalization
(d) `autopct` formats each wedge's percentage label; wedge angles are proportional to each value divided by the total

Q52. [introduction to pandas | Hard]

Why can a pandas `Series` hold heterogeneous data more flexibly than a plain NumPy array of a fixed numeric dtype?

(a) A `Series` stores each element in a separate NumPy array of its own dtype
(b) A `Series` can fall back to the `object` dtype, storing Python objects of mixed types, though at the cost of vectorized performance
(c) A `Series` internally converts all values to strings to allow mixing types
(d) A `Series` cannot hold mixed types; it always raises an error like NumPy

Q53. [operations using dataframes | Hard]

Given `df.groupby('cat')['val'].transform('mean')`, how does its output differ from `df.groupby('cat')['val'].mean(`

(a) `transform` drops the grouping while `mean` keeps every original row
(b) `transform` returns a Series aligned to the original rows (each row gets its group mean); `mean` returns one aggregated value per group
(c) `transform` returns a scalar; `mean` returns a full DataFrame
(d) Both return one value per group with identical shape

Q54. [working with csv files | Hard]

What is the effect of `pd.read_csv('data.csv', index_col=0)` versus omitting `index_col`?

(a) Both produce identical results because the first column always becomes the index
(b) `index_col=0` skips the first column entirely rather than using it as an index
(c) With `index_col=0`, the first column becomes the DataFrame index; without it, a default `RangeIndex` (0,1,2,...) is created and all columns are kept as data
(d) Omitting `index_col` causes the last column to be used as the index by default

Q55. [dataframe | Hard]

Which comparison of `df['col']` and `df[['col']]` is correct?

(a) `df[['col']]` raises an error because double brackets are invalid syntax
(b) `df['col']` returns a 1D Series; `df[['col']]` returns a single-column DataFrame (2D)
(c) `df['col']` returns a DataFrame; `df[['col']]` returns a Series
(d) Both return a Series with identical shape and type

Q56. [multiple subplots in one figure | Hard]

When `fig, axes = plt.subplots(1, 3, sharey=True)` is used, what does `sharey=True` accomplish?

(a) It forces all subplots to share the same x-axis instead of the y-axis
(b) All three subplots share the same y-axis scale and limits, and inner y-tick labels are hidden for clarity
(c) The y-data of the three plots is summed into a single shared plot
(d) Each subplot gets an independent y-axis automatically scaled to its own data

Q57. [series | Hard]

For `s = pd.Series([1, 2, 3])`, why does `s + pd.Series([1, 2, 3], index=[1, 2, 3])` produce `NaN` values?

(a) Labels are aligned before addition; only indices 1 and 2 overlap, so indices 0 and 3 yield `NaN`
(b) The operation fails entirely and returns an empty Series
(c) Addition is positional, so all three pairs add cleanly with no `NaN`
(d) `NaN` appears because the two Series have different lengths

Q58. [histograms | Hard]

In `plt.hist(data, bins=20, density=True)`, what does `density=True` change about the y-axis?

(a) The y-axis shows cumulative counts up to each bin
(b) The y-axis shows raw counts multiplied by the number of bins
(c) The y-axis shows probability density so the total area under the bars equals 1, rather than raw counts
(d) The y-axis is unchanged; `density` only affects bar color

Q59. [line plots | Hard]

Given time-series `x` values that are unsorted, what visual artifact does `plt.plot(x, y)` produce and why?

(a) It raises an error demanding sorted x-values
(b) Matplotlib automatically sorts x before drawing, so the line is always smooth
(c) Only the points are shown with no connecting line at all
(d) The line zig-zags back and forth because `plot` connects points in the order given, not in sorted x-order

Q60. [operations using dataframes | Hard]

What does `df.merge(other, on='key', how='outer')` produce compared to `how='inner'`?

(a) Both keep the same rows; `how` only reorders columns
(b) `outer` keeps only matching keys; `inner` keeps everything
(c) `outer` concatenates rows vertically without matching on keys

(d) `outer` keeps all keys from both frames (filling unmatched sides with `NaN`); `inner` keeps only keys present in both

5.2 Section B: Comprehensive Subjective Problem with Detailed Analytical Rubric

Problem 5 **[Subjective Analytical Problem — 10 Marks]**

1 Construct an end-to-end data processing and visualization pipeline with Pandas and Matplotlib:

(a) Load a sales dataset, impute missing numeric values with column medians, and remove duplicate rows based on composite keys.

(b) Compute monthly aggregated revenue, moving averages, and filter customer segments whose cumulative revenue exceeds the 90th percentile.

(c) Generate a multi-panel Matplotlib publication figure displaying: (1) Time series line plot with confidence bands, (2) Distribution histogram with KDE curve, and (3) Categorical horizontal bar chart.

Step-by-Step Analytical & Algorithmic Solution:

Pandas & Matplotlib Pipeline:

```
 import pandas as pd
import matplotlib.pyplot as plt

def clean_and_aggregate(df):
  df = df.drop_duplicates(subset=['transaction_id', 'customer_id']).copy()
  num_cols = df.select_dtypes(include=['float64', 'int64']).columns
  df[num_cols] = df[num_cols].fillna(df[num_cols].median())
  df['date'] = pd.to_datetime(df['date'])
  monthly = df.resample('M', on='date')['revenue'].sum().reset_index()
  monthly['rolling_3m'] = monthly['revenue'].rolling(window=3).mean()
  return df, monthly

def plot_dashboard(monthly, df):
  fig, axes = plt.subplots(1, 2, figsize=(14, 5))
  axes[0].plot(monthly['date'], monthly['revenue'], label='Monthly Revenue', marker='o')
  axes[0].plot(monthly['date'], monthly['rolling_3m'], label='3M MA', linestyle='-')
  axes[0].set_title("Revenue Dynamics")
  axes[0].legend()
  axes[1].hist(df['revenue'], bins=30, color='skyblue', edgecolor='black')
  axes[1].set_title("Revenue Distribution")
  plt.tight_layout()
  plt.savefig('sales_dashboard.png', dpi=300)
```

Marking Insights: Emphasize correct use of `.loc`, explicit copies to avoid `SettingWithCopyWarning`, and memory optimization through categorical conversions.

- **Algorithmic Formulation (3 Marks):** Correct problem decomposition, function signatures, and parameter documentation.
- **Syntactic Rigor & Implementation (4 Marks):** Valid Python syntax, proper error handling, defensive input validation, and avoidance of anti-patterns.
- **Complexity & Edge Analysis (3 Marks):** Accurate time and space asymptotic analysis $\mathcal{O}(n)$ and testing edge conditions (null inputs, zero division, empty collections).

Chapter 6

Unit 6: Real-World Applications (OpenCV, Pygame, Web Scraping, Toolboxes)

6.1 Section A: Multiple Choice Practice Questions (60 Curated Problems)

This section contains 60 rigorous multiple-choice questions extracted from university course examination assessments. Review each question, test edge-case syntax, and verify code execution mechanics.

Q1. [OpenCV-based image operations | Easy]

Which Python library is primarily used for computer vision and image processing tasks?

(a) Matplotlib
(b) PyGame
(c) OpenCV
(d) BeautifulSoup

Q2. [OpenCV-based image operations | Easy]

Which function is used to read an image from a file in OpenCV?

(a) `cv2.showImage()`
(b) `cv2.loadFile()`
(c) `cv2.openImage()`
(d) `cv2.imread()`

Q3. [OpenCV-based image operations | Easy]

In OpenCV, which color format are images loaded in by default?

(a) HSV
(b) BGR
(c) RGB
(d) CMYK

Q4. [OpenCV-based image operations | Easy]

Which OpenCV function displays an image in a window?

(a) `cv2.view()`
(b) `cv2.render()`
(c) `cv2.imshow()`
(d) `cv2.display()`

Q5. [PyGame-based gaming creation | Easy]

PyGame is a Python library mainly used for creating what type of applications?

(a) Web servers
(b) Machine learning models
(c) Database systems

(d) Games and multimedia

Q6. [PyGame-based gaming creation | Easy]
Which function must be called to start using PyGame's modules?
(a) `pygame.begin()`
(b) `pygame.run()`
(c) `pygame.start()`
(d) `pygame.init()`

Q7. [PyGame-based gaming creation | Easy]
In PyGame, which object represents the main window or drawing surface?
(a) Frame
(b) Panel
(c) Surface
(d) Canvas

Q8. [PyGame-based gaming creation | Easy]
Which PyGame concept is typically used to continuously update the game and handle events?
(a) Data pipeline
(b) Game loop
(c) Event stack
(d) Render tree

Q9. [web-scraping-based url access | Easy]
Which Python library is commonly used to send HTTP requests to access a URL?
(a) `requests`
(b) `turtle`
(c) `numpy`
(d) `pygame`

Q10. [web-scraping-based url access | Easy]
Which library is widely used to parse HTML content during web scraping?
(a) Seaborn
(b) BeautifulSoup
(c) PyGame
(d) OpenCV

Q11. [web-scraping-based url access | Easy]
What does the term "web scraping" mean?
(a) Hosting websites on a server
(b) Extracting data from websites automatically
(c) Designing website layouts
(d) Encrypting web traffic

Q12. [web-scraping-based url access | Easy]
An HTTP status code of 200 typically indicates what?
(a) Access is forbidden
(b) The request succeeded
(c) A server error occurred
(d) The page was not found

Q13. [data visualization | Easy]
Which Python library is one of the most widely used for creating basic plots and charts?
(a) Matplotlib
(b) Requests
(c) PyGame
(d) OpenCV

Q14. [data visualization | Easy]
Which chart type is best suited for showing trends in data over time?
(a) Line chart
(b) Pie chart
(c) Word cloud
(d) Scatter matrix

Q15. [data visualization | Easy]
Which library is built on top of Matplotlib and provides attractive statistical graphics?
(a) BeautifulSoup
(b) Seaborn
(c) Flask
(d) PyGame

Q16. [data visualization | Easy]
Which chart is most appropriate for showing the proportion of parts to a whole?
(a) Histogram
(b) Line chart
(c) Pie chart
(d) Box plot

Q17. [discussion on advanced toolboxes in python | Easy]
Which library is commonly used for numerical computing and working with arrays in Python?
(a) Requests
(b) PyGame
(c) NumPy
(d) Pillow

Q18. [discussion on advanced toolboxes in python | Easy]
Which Python library is primarily used for data analysis and handling tabular data?
(a) OpenCV
(b) Turtle
(c) Pandas
(d) PyGame

Q19. [demonstration of real-world applications | Easy]
Which of the following is a real-world application where Python is commonly used?
(a) Manufacturing car engines
(b) Data analysis and machine learning
(c) Painting walls
(d) Baking bread

Q20. [storytelling | Easy]
In the context of data, what does "data storytelling" mainly involve?
(a) Encrypting datasets securely
(b) Writing fictional novels with code
(c) Deleting outdated records
(d) Communicating insights using data and visuals

Q21. [OpenCV-based image operations | Medium]
In OpenCV, when you read an image using `cv2.imread()`, in what channel order are the pixel values stored by default?
(a) HSV
(b) BGR
(c) Grayscale
(d) RGB

Q22. [OpenCV-based image operations | Medium]

You want to convert a color image `img` to grayscale in OpenCV. Which line accomplishes this?

(a) `gray = cv2.imread(img, 0)`
(b) `gray = cv2.convert(img, GRAY)`
(c) `gray = cv2.grayscale(img)`
(d) `gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`

Q23. [OpenCV-based image operations | Medium]

What does the shape attribute `img.shape` return as `(480, 640, 3)` indicate about the image?

(a) 480 frames, 640 pixels, 3 bits
(b) Height 480, width 640, and 3 color channels
(c) 480 rows, 640 channels, 3 layers
(d) Width 480, height 640, and 3 color channels

Q24. [PyGame-based gaming creation | Medium]

In a typical PyGame program, which function must be called first to start up all imported PyGame modules?

(a) `pygame.start()`
(b) `pygame.init()`
(c) `pygame.run()`
(d) `pygame.begin()`

Q25. [PyGame-based gaming creation | Medium]

Inside a PyGame main loop, which call is responsible for actually updating the full screen with everything drawn?

(a) `pygame.display.init()`
(b) `pygame.screen.draw()`
(c) `pygame.display.flip()`
(d) `pygame.event.get()`

Q26. [PyGame-based gaming creation | Medium]

Why is `pygame.time.Clock()` combined with `clock.tick(60)` used in a game loop?

(a) To limit the loop to a maximum of 60 frames per second
(b) To pause the game for 60 seconds
(c) To set the background refresh color
(d) To count 60 total game events

Q27. [PyGame-based gaming creation | Medium]

In PyGame, how do you typically detect that the user clicked the window's close button?

(a) Call `pygame.display.quit()` repeatedly
(b) Check for `event.type == pygame.QUIT` in the event loop
(c) Wait for `pygame.KEYDOWN`
(d) Check `pygame.window.closed()`

Q28. [web-scraping-based url access | Medium]

When scraping a webpage in Python, which combination is most commonly used to fetch the HTML and then parse it?

(a) `urllib` to parse, `requests` to fetch
(b) `BeautifulSoup` to fetch, `requests` to parse
(c) `json` to fetch, `csv` to parse
(d) `requests` to fetch, `BeautifulSoup` to parse

Q29. [web-scraping-based url access | Medium]

After `response = requests.get(url)`, which attribute do you check to confirm the request succeeded (HTTP 200)?

(a) `response.success`
(b) `response.ok_flag`
(c) `response.result`

(d) `response.status_code`

Q30. [web-scraping-based url access | Medium]

Using BeautifulSoup, which method returns a list of all `<a>` anchor tags on a page?

(a) `soup.find('a')`
(b) `soup.get('a')`
(c) `soup.find_all('a')`
(d) `soup.select_one('a')`

Q31. [web-scraping-based url access | Medium]

A website's `robots.txt` disallows scraping certain paths. What is the correct interpretation for a responsible scraper?

(a) It signals which paths crawlers should not access, and should be respected
(b) It automatically blocks all Python requests
(c) It encrypts the page so it cannot be parsed
(d) It is only relevant to search engines and can be ignored

Q32. [data visualization | Medium]

In Matplotlib, which plot type is most appropriate for showing the relationship between two continuous numeric variables?

(a) Pie chart
(b) Histogram of one variable
(c) Scatter plot
(d) Bar chart of categories

Q33. [data visualization | Medium]

You want to visualize the distribution and frequency of a single continuous variable such as exam scores. Which chart is best?

(a) Histogram
(b) Pie chart
(c) Line chart
(d) Scatter plot

Q34. [data visualization | Medium]

In Matplotlib, what is the purpose of calling `plt.show()` at the end of a plotting script?

(a) It clears all existing plots
(b) It saves the figure to disk
(c) It renders and displays the figure window
(d) It sets the figure resolution

Q35. [data visualization | Medium]

Which library is built on top of Matplotlib and provides high-level functions for attractive statistical graphics?

(a) Requests
(b) NumPy
(c) Pillow
(d) Seaborn

Q36. [discussion on advanced toolboxes in python | Medium]

Which Python library is primarily used for numerical computing with efficient multi-dimensional array operations?

(a) BeautifulSoup
(b) NumPy
(c) PyGame
(d) Flask

Q37. [discussion on advanced toolboxes in python | Medium]

A data analyst needs to load a CSV file into a labeled, tabular data structure with columns. Which library and object are most appropriate?

(a) `pandas` DataFrame
(b) `numpy` scalar
(c) `requests` Session
(d) `matplotlib` Figure

Q38. [discussion on advanced toolboxes in python | Medium]
Which pair correctly matches a Python toolbox to its main purpose?
(a) scikit-learn → machine learning
(b) Flask → image processing
(c) TensorFlow → web scraping
(d) Pandas → 3D game rendering

Q39. [demonstration of real-world applications | Medium]
A team builds a Python system that detects faces in a live camera feed. Which library is the natural core of this application?
(a) OpenCV
(b) Pandas
(c) BeautifulSoup
(d) Flask

Q40. [demonstration of real-world applications | Medium]
A company wants to automatically collect daily product prices from several e-commerce sites into a spreadsheet for analysis. Which workflow best fits?
(a) Read the images with OpenCV, then train scikit-learn
(b) Scrape pages with requests/BeautifulSoup, then store in a pandas DataFrame
(c) Render each site with PyGame, then plot with Seaborn
(d) Fetch prices with Matplotlib, then save with NumPy

Q41. [OpenCV-based image operations | Hard]
In OpenCV, you load an image with `cv2.imread('img.png')` and display it using Matplotlib's `plt.imshow()`. The colors appear distorted (blues look red). What is the root cause and correct fix?
(a) Matplotlib requires normalization; divide the array by 255 before display
(b) OpenCV loads images in BGR order while Matplotlib expects RGB; convert using `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`
(c) The image uses a wrong color depth; convert to grayscale first with `cv2.COLOR_BGR2GRAY`
(d) The image is corrupted; reload it with `cv2.IMREAD_UNCHANGED`

Q42. [OpenCV-based image operations | Hard]
You apply a 3×3 averaging (box) blur kernel to an image, then apply the same kernel again. Compared to a single 5×5 box blur, the two-pass 3×3 approach:
(a) Produces a smoothing effect similar to a $5 \times$
(b) Produces less smoothing than a single $3 \times$
(c) Produces exactly the same result as a single $5 \times$
(d) Cannot be performed because kernels are not associative

Q43. [OpenCV-based image operations | Hard]
When performing edge detection with `cv2.Canny(img, threshold1, threshold2)`, what best describes the role of the two thresholds in hysteresis?
(a) `threshold1` sets Gaussian blur size and `threshold2` sets the gradient magnitude
(b) Pixels above `threshold2` are strong edges; pixels between the thresholds become edges only if connected to strong edges
(c) Both thresholds must be exceeded simultaneously for a pixel to be an edge
(d) Pixels below `threshold1` are strong edges; those above `threshold2` are discarded as noise

Q44. [PyGame-based gaming creation | Hard]
In a PyGame loop, a developer moves a sprite by adding a fixed number of pixels each frame without using `clock.tick()`. On faster hardware the object moves too quickly. What is the correct fix?

(a) Call `pygame.display.flip()` twice per loop iteration
(b) Increase the fixed pixel step to compensate for the frame rate
(c) Multiply movement by delta time (seconds since last frame) so motion is frame-rate independent
(d) Reduce the sprite's image resolution to slow rendering

Q45. [PyGame-based gaming creation | Hard]
A PyGame program's window becomes unresponsive and shows "Not Responding" during gameplay. Which omission most likely causes this?

(a) Sprites are drawn before the screen is filled
(b) The display was initialized with too small a resolution
(c) The event queue is never processed with `pygame.event.get()` or `pygame.event.pump()`
(d) The frame rate is capped too low with `clock.tick(30)`

Q46. [PyGame-based gaming creation | Hard]
For pixel-perfect collision between two moving sprites in PyGame, which approach is most accurate but computationally heavier than rectangle collision?

(a) Checking Euclidean distance between sprite centers
(b) Using `pygame.sprite.spritecollide` with `collided=None`
(c) Mask-based collision using `pygame.mask.from_surface` and `mask.overlap`
(d) Comparing `rect.colliderect` on both sprites

Q47. [web-scraping-based url access | Hard]
You scrape a page with `requests.get(url)` and BeautifulSoup, but the target data is missing from the returned HTML even though it appears in the browser. What is the most probable reason?

(a) The content is rendered client-side by JavaScript after page load, which `requests` does not execute
(b) BeautifulSoup cannot parse `<div>` tags without the `lxml` parser
(c) `requests` strips all dynamic tags from the response by default
(d) The server returned a 200 status so the data must be present in a hidden attribute

Q48. [web-scraping-based url access | Hard]
Which statement about `robots.txt` and web scraping is most accurate?

(a) It is an advisory standard that requests crawlers avoid certain paths, but it does not technically enforce access control
(b) It is a firewall rule that blocks all unauthorized scraping requests
(c) It automatically rate-limits any client that ignores it
(d) It encrypts URLs so scrapers cannot read the site structure

Q49. [web-scraping-based url access | Hard]
A scraper repeatedly hits a site and starts receiving HTTP 429 responses. Which combination best addresses this while remaining respectful?

(a) Switch every request to a new random User-Agent and ignore the status code
(b) Send requests faster to finish before the block takes effect
(c) Disable SSL verification to bypass the rate limiter
(d) Add delays between requests and implement exponential backoff honoring the `Retry-After` header

Q50. [data visualization | Hard]
You plot data spanning 1 to 1 0 6 on a linear y-axis and small values become invisible near zero. Which change best reveals structure across all magnitudes?

(a) Increase the figure DPI to 300
(b) Use a logarithmic y-axis (e.g., `plt.yscale('log')`)
(c) Add more gridlines to the linear axis
(d) Switch the plot color to a brighter palette

Q51. [data visualization | Hard]
When designing a chart for viewers who may be color-blind, which practice is most effective?

(a) Combine a color-blind-safe palette with redundant encodings like markers, line styles, or direct labels
(b) Rely on a rainbow (jet) colormap for continuous data

(c) Remove the legend so viewers focus on colors
(d) Use only red and green to maximize contrast

Q52. [data visualization | Hard]
A bar chart of average sales per region hides the fact that one region has a bimodal distribution. Which visualization better exposes this within-group structure?
(a) A violin plot or box plot showing the distribution shape per region
(b) A pie chart of the same averages
(c) A line chart connecting the regional means
(d) A single stacked bar of all regions combined

Q53. [data visualization | Hard]
In Matplotlib, what is the key difference between the stateful `pyplot` interface and the object-oriented (Axes) interface for complex multi-panel figures?
(a) The `pyplot` interface renders faster because it caches figures
(b) Only the `pyplot` interface supports saving figures to file
(c) The OO interface gives explicit control over each `Axes` object, avoiding ambiguity about which subplot commands affect
(d) The OO interface cannot create subplots without `plt.figure()`

Q54. [discussion on advanced toolboxes in python | Hard]
You perform an element-wise operation on two large arrays. Why is NumPy vectorization typically much faster than a Python `for` loop?
(a) Python `for` loops cannot operate on numeric data
(b) NumPy stores data on the GPU by default
(c) NumPy executes the operation in compiled C code over contiguous memory, avoiding per-element Python interpreter overhead
(d) NumPy loops are automatically parallelized across all CPU cores by default

Q55. [discussion on advanced toolboxes in python | Hard]
A Pandas operation `df[df['a'] > 0]['b'] = 5` produces a `SettingWithCopyWarning` and the change may not persist. What is the correct fix?
(a) Disable the warning with `pd.options.mode.chained_assignment = None`
(b) Use `.loc` for combined selection and assignment: `df.loc[df['a'] > 0, 'b'] = 5`
(c) Convert the column to a list, modify it, and reassign
(d) Wrap the expression in `df.copy()` before assigning

Q56. [discussion on advanced toolboxes in python | Hard]
When training a scikit-learn model, why should scaling (e.g., `StandardScaler`) be fit only on the training data and not on the full dataset before splitting?
(a) Fitting on all data leaks test-set statistics into training, producing overly optimistic evaluation
(b) The scaler cannot process both sets simultaneously due to memory limits
(c) Test data must never be numerically transformed
(d) Scaling the test set is impossible without the training labels

Q57. [storytelling | Hard]
In data storytelling, a stakeholder audience needs to grasp one key takeaway quickly. Which approach best supports this goal?
(a) Remove all text so the numbers speak for themselves
(b) Present every chart produced during analysis in chronological order
(c) Maximize data density by overlaying all metrics on one axis
(d) Lead with a clear headline insight and use annotations to direct attention to the supporting evidence

Q58. [storytelling | Hard]
A chart truncates its y-axis to start at 95 instead of 0 , making a 2% change look dramatic. In the context of ethical storytelling, this is best described as:
(a) A required convention for all bar charts

(b) A best practice that improves resolution and is always appropriate
(c) A misleading design choice that exaggerates differences and should be disclosed or avoided
(d) An error that Matplotlib prevents automatically

Q59. [demonstration of real-world applications | Hard]

You must process a live video stream, detect motion, and log events to a dashboard. Which combination of Python tools maps most appropriately to each subtask?

(a) Requests for capture, scikit-learn for logging, and PyGame for the dashboard
(b) OpenCV for frame capture/motion detection, Pandas for event logging, and Matplotlib/Plotly for the dashboard
(c) Matplotlib for capture, OpenCV for logging, and Requests for the dashboard
(d) PyGame for capture, BeautifulSoup for motion detection, and NumPy for the dashboard

Q60. [demonstration of real-world applications | Hard]

A pipeline scrapes prices hourly, stores them, and predicts trends. To keep it maintainable and avoid re-scraping unchanged pages, which design choice is most sound?

(a) Combine scraping, storage, and modeling into one function for simplicity
(b) Store all data only in memory to speed up predictions
(c) Cache responses with conditional requests (ETag/If-Modified-Since) and separate scraping, storage, and modeling into modular stages
(d) Scrape every page fully each run regardless of changes to ensure freshness

6.2 Section B: Comprehensive Subjective Problem with Detailed Analytical Rubric

Problem 6 **[Subjective Analytical Problem — 10 Marks]**

1 Design an applied engineering pipeline integrating OpenCV image processing and web API data retrieval:

(a) Write an OpenCV script that reads an image, converts BGR to Grayscale, applies Gaussian Blur with 5×5 kernel to eliminate high-frequency noise, detects edges using Canny edge detector ($T_{\text{lower}} = 50, T_{\text{upper}} = 150$), and finds contours.

(b) Implement a resilient Python web scraper using `requests` and `BeautifulSoup` that extracts real-time product prices from an e-commerce page, handles HTTP 429 rate limiting with exponential backoff, and saves structured records into JSON format.

Step-by-Step Analytical & Algorithmic Solution:

OpenCV & Web Scraping Implementations:

```
 import cv2
import time, requests, json
from bs4 import BeautifulSoup

def process_image_contours(image_path):
  img = cv2.imread(image_path)
  if img is None:
    raise FileNotFoundError(f"Cannot read image at {image_path}")
  gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
  blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)
  edges = cv2.Canny(blurred, 50, 150)
  contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
  return len(contours), edges

def fetch_with_backoff(url, max_retries=3):
  headers = {"User-Agent":  "AcademicResearcher/1.0"}
  for attempt in range(max_retries):
    resp = requests.get(url, headers=headers)
    if resp.status_code == 200:
      soup = BeautifulSoup(resp.text, 'html.parser')
      return soup
    elif resp.status_code == 429:
      time.sleep(2 ** attempt)
  raise RuntimeError("Max retries exceeded or forbidden")
```

Engineering Rubric: Highlight proper channel ordering (BGR vs RGB), threshold selection rationale for Canny suppression, header spoofing ethics, and robots.txt compliance.

- **Algorithmic Formulation (3 Marks):** Correct problem decomposition, function signatures, and parameter documentation.
- **Syntactic Rigor & Implementation (4 Marks):** Valid Python syntax, proper error handling, defensive input validation, and avoidance of anti-patterns.
- **Complexity & Edge Analysis (3 Marks):** Accurate time and space asymptotic analysis $\mathcal{O}(n)$ and testing edge conditions (null inputs, zero division, empty collections).